

LECTURE 3

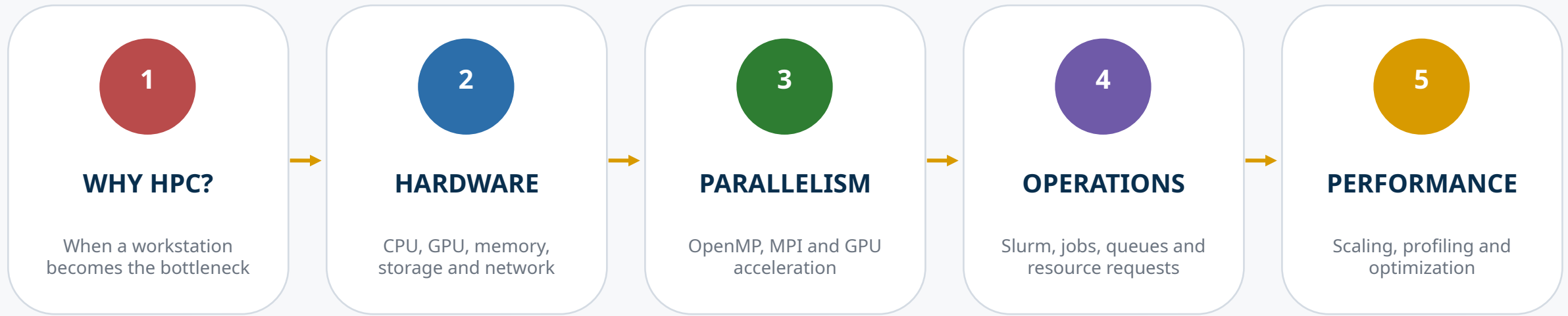
High-Performance Computing for Scientific Modeling and Simulation

From Workstations to Supercomputers
Speaker: Prof. Dr. Tassadaq Hussain

Understand the hardware, parallel-computing concepts, software stack and scaling principles that make large scientific simulations possible.

Lecture Roadmap

From one processor to a complete HPC environment.



Goal: choose the right resources, run the simulation correctly, and understand whether more hardware will actually make it faster.

Why Does a Simulation Need HPC?

Scientific accuracy often means more data, more resolution, more equations and more time steps.

Finer mesh

More cells/elements

More particles

More atoms/molecules

More physics

Coupled equations

More scenarios

Sweeps/ensembles

Longer runs

More time steps

Computational Work ↑ Memory Requirement ↑ Data Movement ↑ Runtime ↑

HPC lets us divide the work across many processors so that the scientific problem remains practical.

WHY DO SIMULATIONS NEED HPC?

From Real Problem to Simulation to Massive Computation

1. REAL-WORLD PROBLEM

Example:
Aerodynamic analysis of a car to reduce drag and improve fuel efficiency




Lower Drag
Better Performance


Fuel Efficiency
Cost Savings


Lower Emissions
Environmental Impact


Safety & Comfort
Design Validation

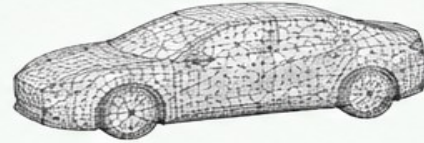

Time to Market
Faster Development

2. MODEL SETUP

Geometry



Mesh (Finer = More Detail)



Coarse Mesh → Finer Mesh

Particles / Cells



Larger Size → Smaller Size

Physical Model

- Governing Equations
- Navier-Stokes Equations
 - Turbulence Model (k- ω SST)
 - Energy Equation (optional)
 - Boundary Conditions

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p + \nabla \cdot (\mu \nabla \mathbf{u}) + \mathbf{f}$$

Simulation Setup

Inlet Velocity
Outlet Pressure
Wall Conditions
Fluid Properties
...

3. WHAT THIS CREATES



Millions to Billions of Cells
Finer mesh



Millions to Billions of Particles
Smaller particle size



Complex Physics
Coupled equations & models



Huge Computations
Many operations per time step



Large Memory Requirement
Store matrices, variables, results



Long Run Time
Thousands to millions of time steps

Computational Work ↑

Memory Requirement ↑

Data Movement ↑

Run Time ↑

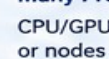
4. HPC COMPUTATION APPROACH

Domain Decomposition

Split the problem into many parts



Distribute Across Many Processors



CPU/GPU cores or nodes

Parallel Computation

Each processor solves its part simultaneously



Communicate & Synchronize

Exchange data between processors



Aggregate Results

Combine all parts to get the full solution



HPC makes large problems possible by solving many small parts at the same time.

5. RESULTS & INSIGHTS



Pressure Distribution & Flow Field



Drag, Lift & Force Coefficients



Heat Transfer (if applicable)



Design Validation & Optimization



Better Product Performance

FROM REAL-WORLD OBSERVATIONS TO SUPERCOMPUTING

1

REAL-WORLD OBSERVATIONS

800 MILLION OBSERVATIONS / DAY

Satellites, stations, aircraft, ships and buoys observe the atmosphere.



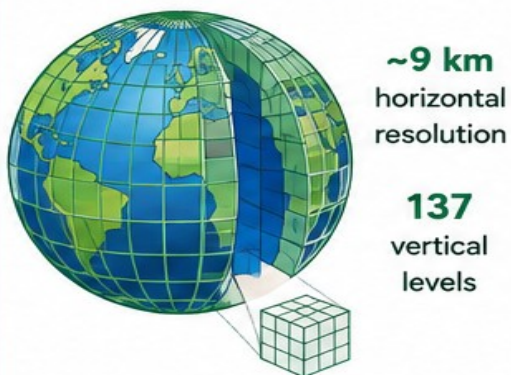
What we measure

2

MATHEMATICAL MODEL

452 BILLION SIMULATED PARAMETERS

Observations initialize the model. Equations are solved across the globe at ~9 km spacing and 137 vertical levels.



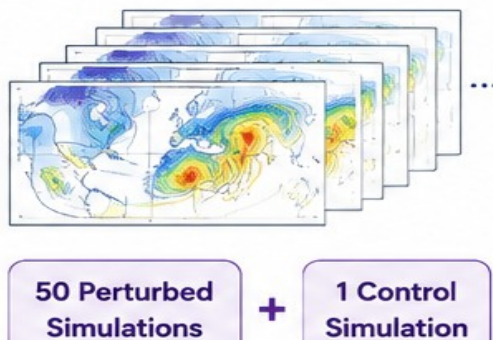
How we represent & solve

3

MODELING & SIMULATION

51 POSSIBLE FUTURES

50 perturbed simulations + 1 control simulation. Predict possible outcomes up to 15 days and quantify uncertainty.



What could happen

4

HPC / SUPERCOMPUTING

MASSIVE COMPUTATION

Run all simulations fast enough for operational forecasts and timely decisions.



COMPUTATIONAL REQUIREMENT PER DAY

~30 PFLOPS-DAY

≈ 30 QUADRILLION CALCULATIONS PER SECOND SUSTAINED OVER 24 HOURS

How we compute at scale



1. OBSERVATIONS

Measure the current state of the Earth system.



2. MODEL

Convert observations into a global model and solve physical laws.



3. SIMULATIONS

Run many scenarios to estimate probability and forecast uncertainty.



4. HPC

Perform trillions of calculations to deliver forecasts on time.



INFORMED DECISIONS

Safer lives, stronger economies and a more resilient future.

REFERENCES



1. ECMWF – Observations

ecmwf.int/en/research/data-assimilation/observations

2. ECMWF – Fifty Years of Earth System Modelling

ecmwf.int/en/elibrary/81651-fifty-years-of-earth-system-modelling-at-ecmwf

3. ECMWF – Medium-Range Ensemble Forecasts

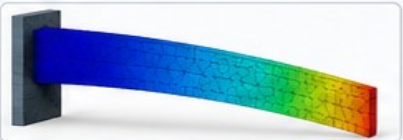
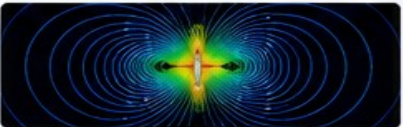
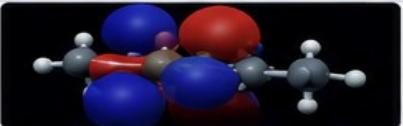
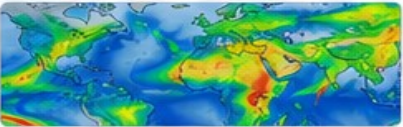
confluence.ecmwf.int/display/FUG/Section+2A.1.2.1+Medium+Range+Ensemble+forecasts

4. ECMWF – Supercomputing at ECMWF

ecmwf.int/en/about/media-centre/focus/2022/fact-sheet-supercomputing-ecmwf

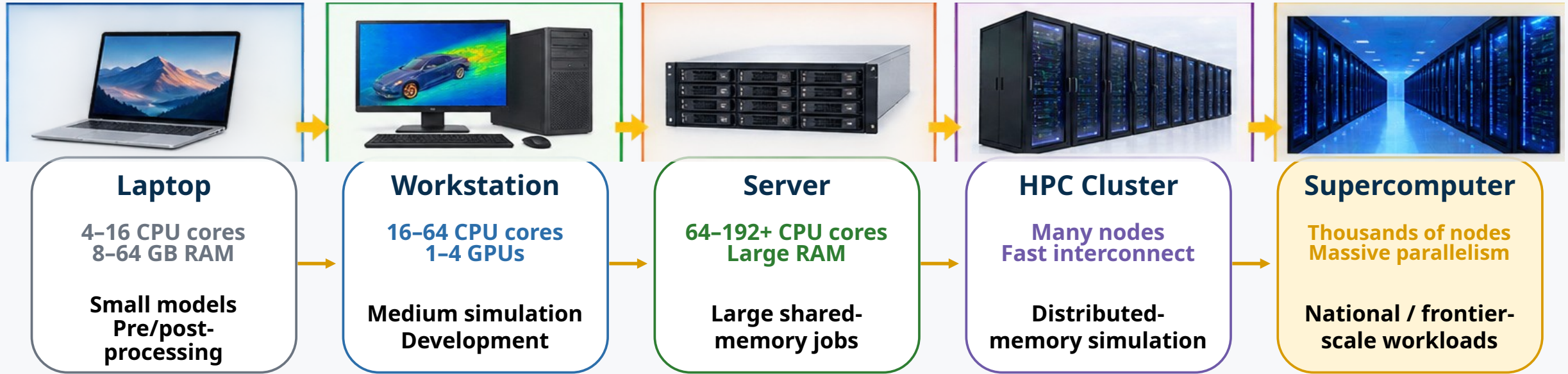


From 800 million observations every day to 452 billion simulated parameters, 51 possible futures, and ~30 PFLOPS-DAY of computing – this is how high-end modeling and simulation turn data into decisions that matter.

Simulation Type		What is Calculated Repeatedly?	Typical Computational Scaling	Rough Computation per Serious Run
	N-Body Simulation 	Force between particles/bodies	Direct: $O(N^2)$; tree methods: $O(N \log N)$	$10^{12} - 10^{18} + \text{FLOP}$
	Molecular Dynamics (MD) 	Forces + new positions of atoms	Usually near $O(N)$ with neighbor lists; long-range methods add cost	$10^{13} - 10^{19} + \text{FLOP}$
	CFD – Fluid Simulation 	Pressure, velocity, turbulence, temperature in every cell	Roughly cells $\times$ equations $\times$ solver iterations $\times$ time steps	$10^{12} - 10^{18} + \text{FLOP}$
	FEA – Structural Simulation 	Displacement, stress, strain at mesh elements/nodes	Assembly + large sparse linear/nonlinear solves	$10^{11} - 10^{17} + \text{FLOP}$
	Electromagnetic Simulation 	Electric & magnetic fields over grid/mesh	Grid cells $\times$ field equations $\times$ time/frequency steps	$10^{12} - 10^{18} + \text{FLOP}$
	Quantum / DFT Simulation 	Electronic wave functions and energies	Conventional plane-wave DFT often approximately $O(N_e^3)$	$10^{13} - 10^{19} + \text{FLOP}$
	Weather / Climate Simulation 	Atmosphere/ocean equations over global grid	Grid cells $\times$ variables $\times$ equations $\times$ time steps $\times$ ensemble members	$10^{15} - 10^{20} + \text{FLOP}$
	Monte Carlo Simulation 	Same calculation with many random samples	$O(N_{\text{samples}} \times C_{\text{sample}})$	$10^9 - 10^{18} + \text{FLOP}$
	Agent-Based Simulation 	Behavior/interactions of individual agents	Agents $\times$ interactions $\times$ time steps	$10^{10} - 10^{16} + \text{FLOP}$

From a Laptop to a Supercomputer

HPC is a continuum of increasingly parallel computing resources.



Approximate Computational Performance (Peak Theoretical Performance)



The key change is not only “more processors” — it is the need to coordinate processors, memory and data across machines.

Inside One Compute Node (Shared Memory Architecture)

A node is a server—the basic building block of a cluster.

COMPUTE NODE

CPU

General-purpose cores
Control + serial work
OpenMP / MPI ranks

RAM

Working memory for
arrays, meshes,
coordinates and solver
state.

GPU

Thousands of lightweight
cores for highly parallel
numerical kernels.

LOCAL I/O

SSD / scratch plus network
access to shared storage.

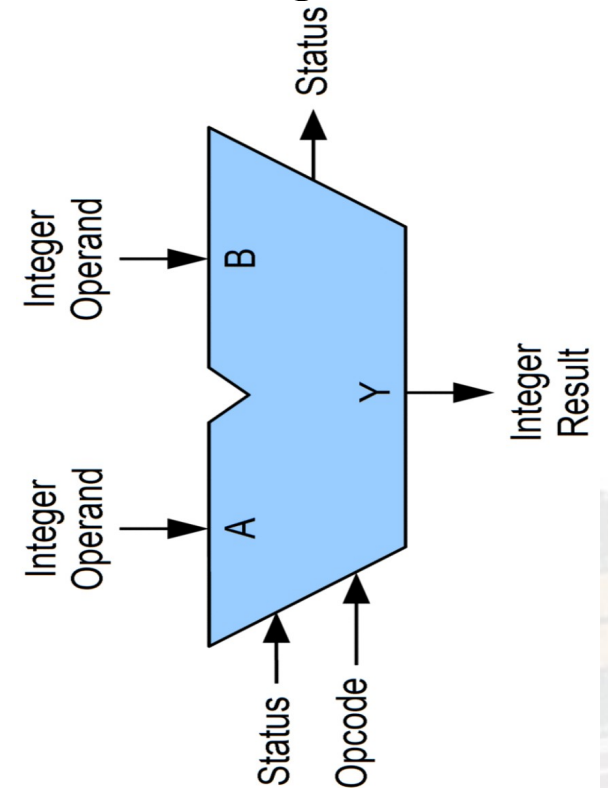
Node-level performance depends on balance: compute capability is wasted if memory or data movement cannot feed it.

Processor

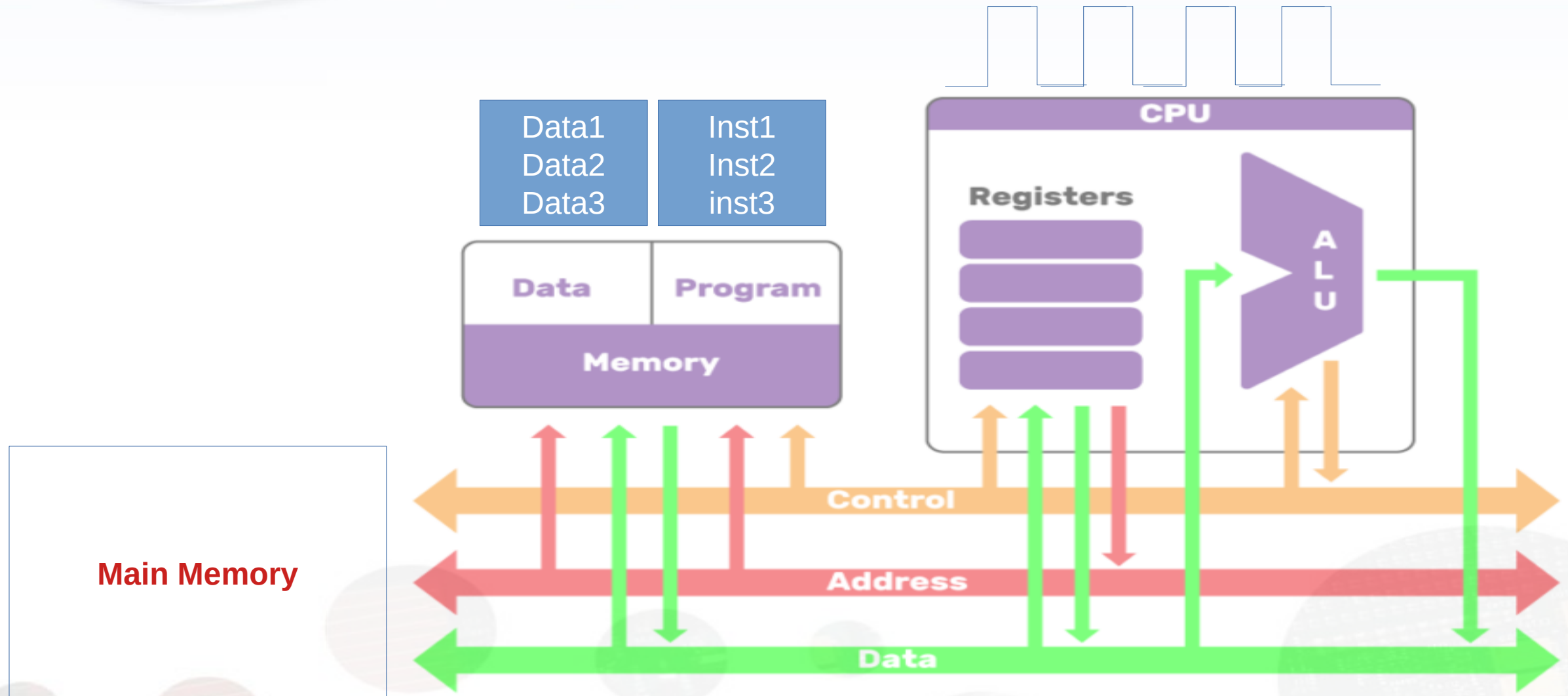
A simple processor takes a single instruction and generate results in a given time called instruction cycles.

An instruction includes two values (operands) and an arithmetic or a logic operation (operator).

Values (operands) can be from memory or peripherals.



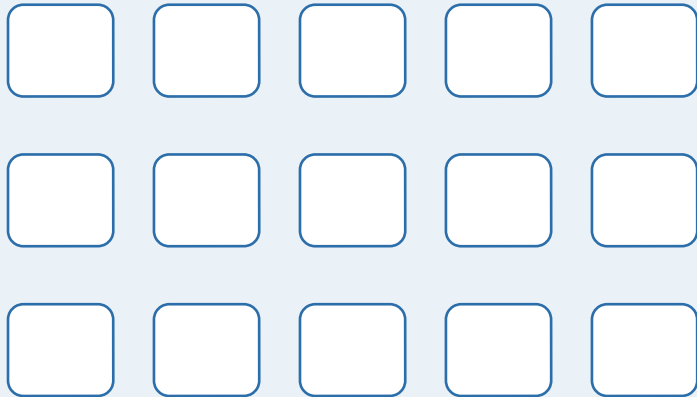
How a CPU Executes a Simulation Calculation



CPU: Flexible General-Purpose Compute

Best for control-heavy code, complex logic and moderately parallel workloads.

CPU



Dozens of powerful cores

Strengths

- Strong single-thread performance
- Large caches
- Flexible branching

Common use

- MPI ranks
- OpenMP threads
- Pre/post-processing

Good fit

- Sparse / irregular algorithms
- Control-heavy kernels

Limitation

- Fewer arithmetic units than a GPU for massively parallel kernels.

Processors for Supercomputers

Lecture 2: Processor Architecture

PROCESSOR TYPE

COMPUTATION CAPABILITY FOR MODELING & SIMULATION



Microprocessor development directions:

- Increasing of clock frequency and speed instruction stream processing
- Processing of large collection of data in single processor instruction - SIMD
- Control path multiplication - multithreading



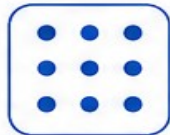
High performance for general-purpose & data-intensive simulations

- Solving large systems of equations
- Data analytics & model calibration
- Multi-threaded workloads & workflows



Vector processors

- NEC SX-6
- Cray (Cray X1)



High-throughput linear algebra & array-based computations

- Matrix operations, FFTs, solvers
- Computational fluid dynamics (CFD)
- Weather & climate modeling



RISC processors

- MIPS
- IBM Power4
- Alpha



Efficient execution of complex simulation codes

- Finite element analysis (FEA)
- Structural & mechanical modeling
- Large-scale scientific applications



CISC processors

- IA32
- AMD x86-64



Balanced performance for enterprise & simulation workloads

- Engineering simulations
- Discrete event simulations
- Pre/post-processing & visualization



VLIW processors

- IA64



Instruction-level parallelism for compute-heavy models

- Physics-based simulations
- Signal & image processing
- High-performance scientific computing

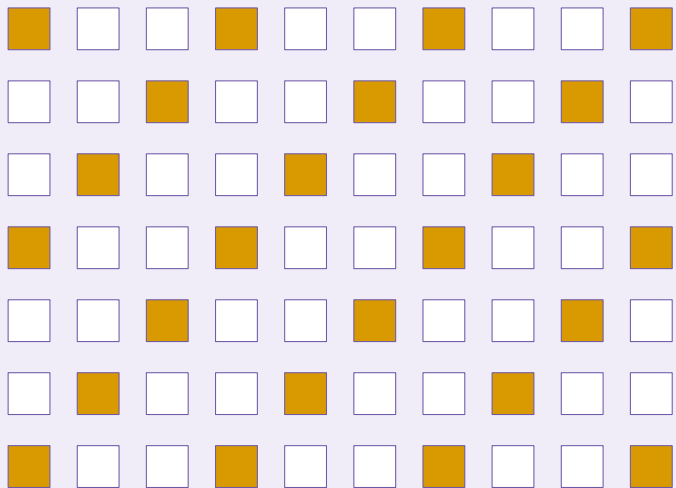


Together, these processors enable scalable, accurate, and efficient modeling & simulation across scientific, engineering, and data-intensive applications.

GPU: Massive Parallel Numerical Throughput

Best when the same kind of mathematical operation can be performed on many data elements at once.

GPU



Thousands of lightweight execution units

Strengths

- High arithmetic throughput
- High memory bandwidth
- Massive parallelism

Common use

- MD force kernels
- Dense linear algebra
- AI / tensor operations

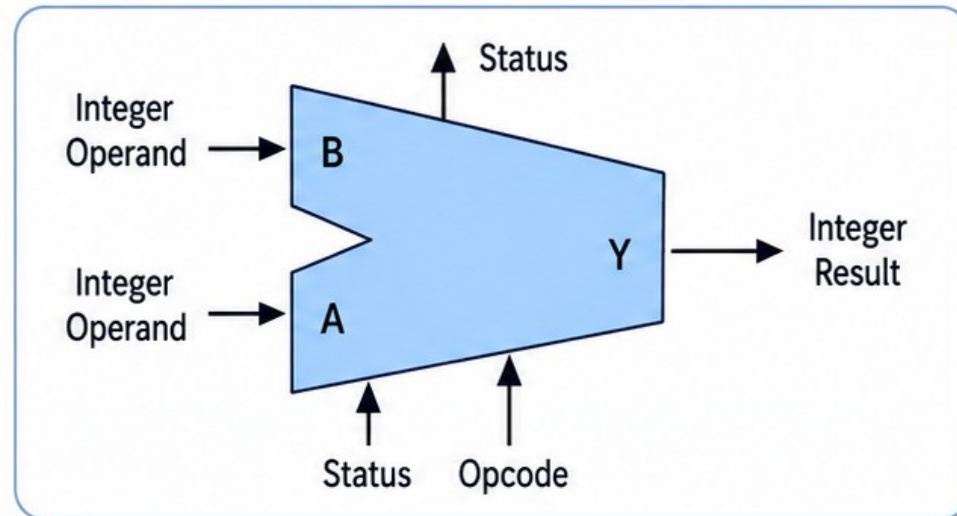
Good fit

- Large regular workloads with many independent operations.

Limitation

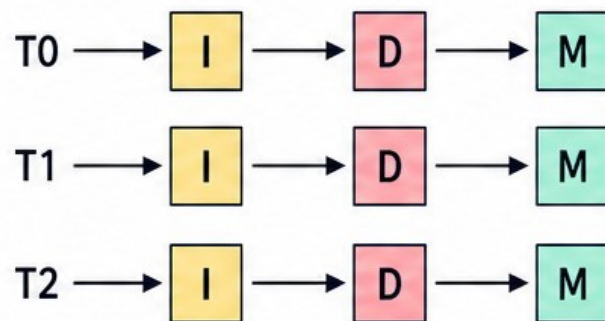
- Performance falls when data movement, branching or serial sections dominate.

Processor Architectures



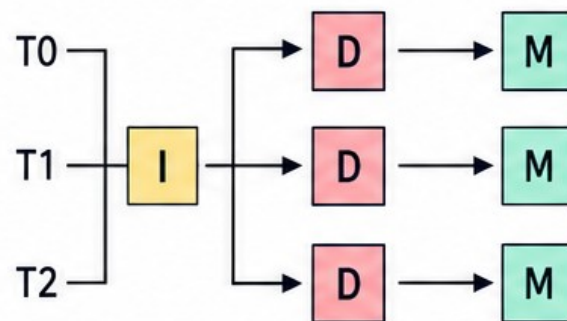
MIMD

Multiple Instruction, Multiple Data



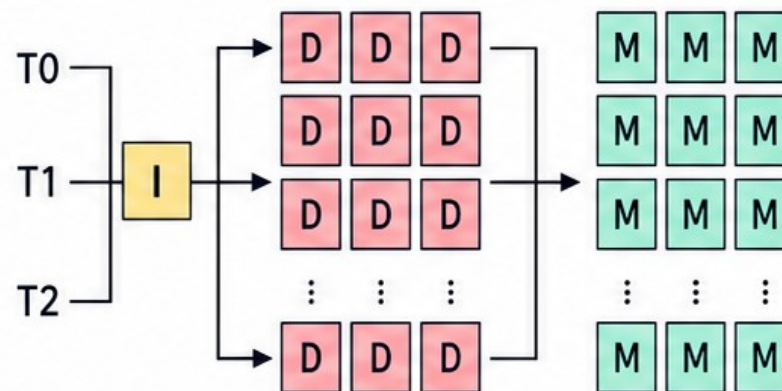
SIMD

Single Instruction, Multiple Data



GPU

Massively Parallel



Instruction



Data



Memory

T

Task/Thread

SIMULATION MODEL

System Memory & Compute Hierarchy

Simulation Model: A simplified representation of a real system that captures the behavior of data flow and computation across the memory hierarchy and compute units.

COMPONENTS

Disk (Storage)
Non-volatile storage.
High capacity, very high latency.

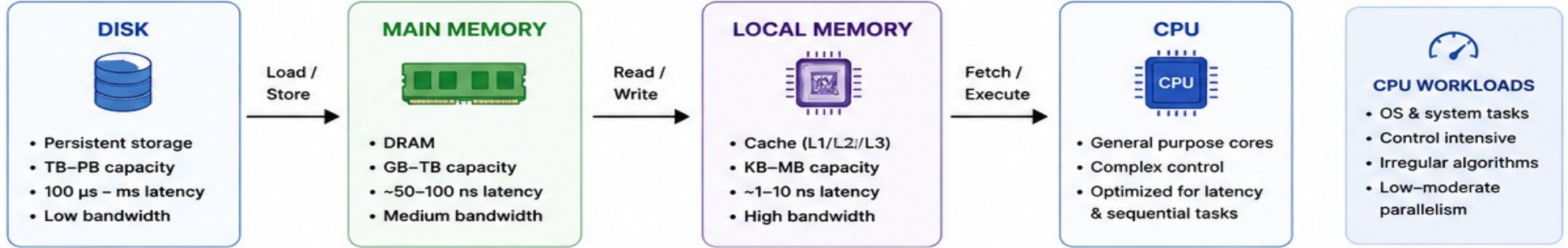
Main Memory
Large capacity, shared
by all processors
and accelerators.

Local Memory
On-chip memory
(Cache / SRAM /
Scratchpad).
Low latency.

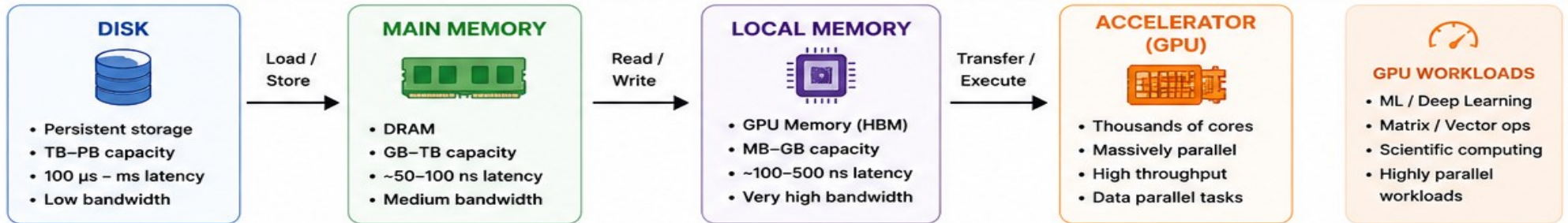
CPU
General purpose
processor for
sequential &
parallel tasks.

Accelerator (GPU)
Specialized many-core
processor for massively
parallel workloads.

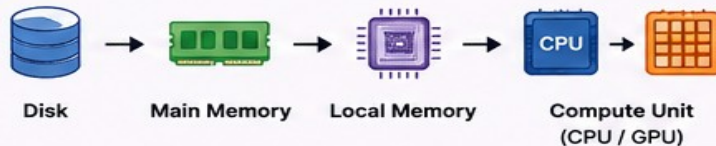
PATH 1: CPU EXECUTION MODEL



PATH 2: ACCELERATOR (GPU) EXECUTION MODEL



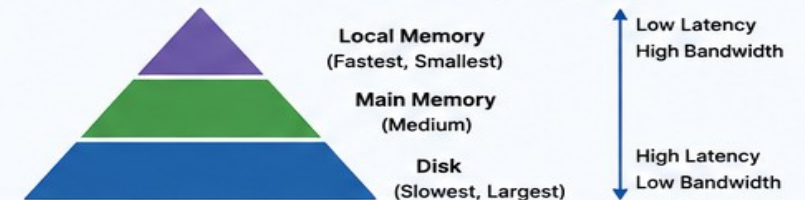
DATA FLOW OVERVIEW



KEY POINTS

- ✓ Data moves from high latency (Disk) to low latency (Compute).
- ✓ Each level closer to compute is faster but smaller.
- ✓ CPU path optimized for low latency & complex control.
- ✓ GPU path optimized for high throughput & parallelism.

MEMORY HIERARCHY (Latency vs Capacity)



CPU vs GPU: Mathematical Computation Inside

Different **architectures**. Different **math strengths**. Built for different problems.

CPU (Central Processing Unit)



Optimized for
low latency



Complex control flow



Handles a wide range
of tasks

VS

GPU (Graphics Processing Unit)



Optimized for
high throughput



Massively parallel
computation



Executes **many** operations
at once



COMPUTATION
MODEL

Few powerful cores



Thousands of cores



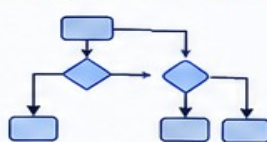
$f(x)$

MATH
STRENGTH

Scalar Math



Complex Operations



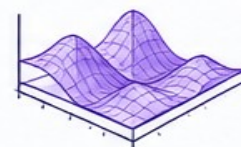
Irregular Problems



Vector & Matrix Math



Linear Algebra



Large Scale Problems

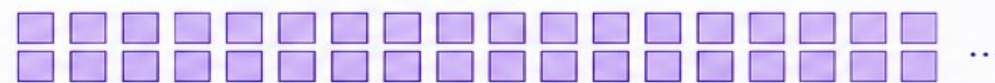


PARALLELISM

Limited Parallelism



Massive Parallelism



BEST AT



Complex
Logic



Databases &
Systems



General
Applications



Low-Latency
Tasks



Matrix
Math



AI / Machine
Learning



Simulations &
Scientific



Graphics &
Rendering

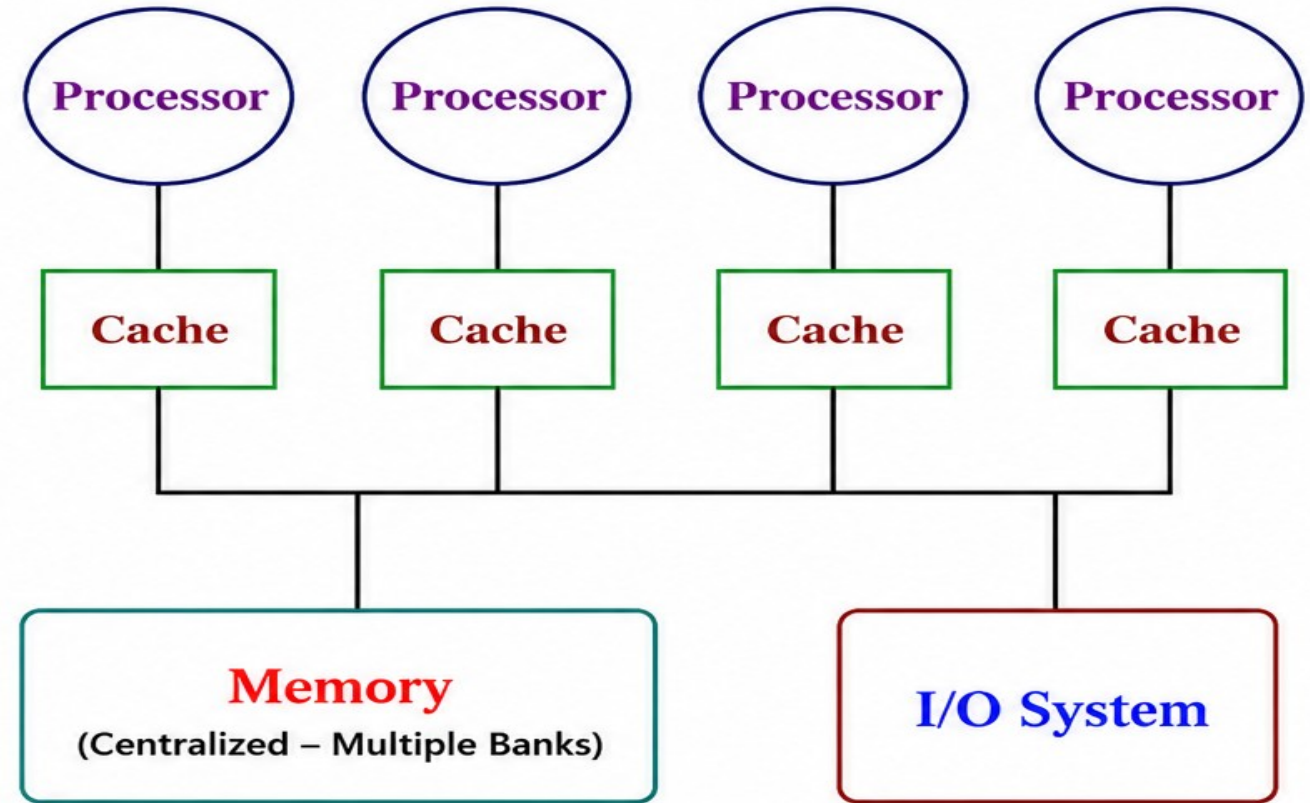


KEY TAKEAWAY: CPUs are optimized for doing **a few things** fast.

GPUs are optimized for doing **many things** at once.

SMP: Shared Memory Processor

- **Small number of similar processors** (at most a few dozen).
- Each processor has a **large cache**.
- A **centralized memory** (multiple banks) is shared through a memory bus.
- Each memory location has **identical access time** from each processor.



All processors share the same memory
with equal access time.

Distributed Memory

Memory is physically distributed among processors and connected through a high-speed interconnection.



Larger processor count.

Supports a large number of processors scaling to hundreds or thousands.

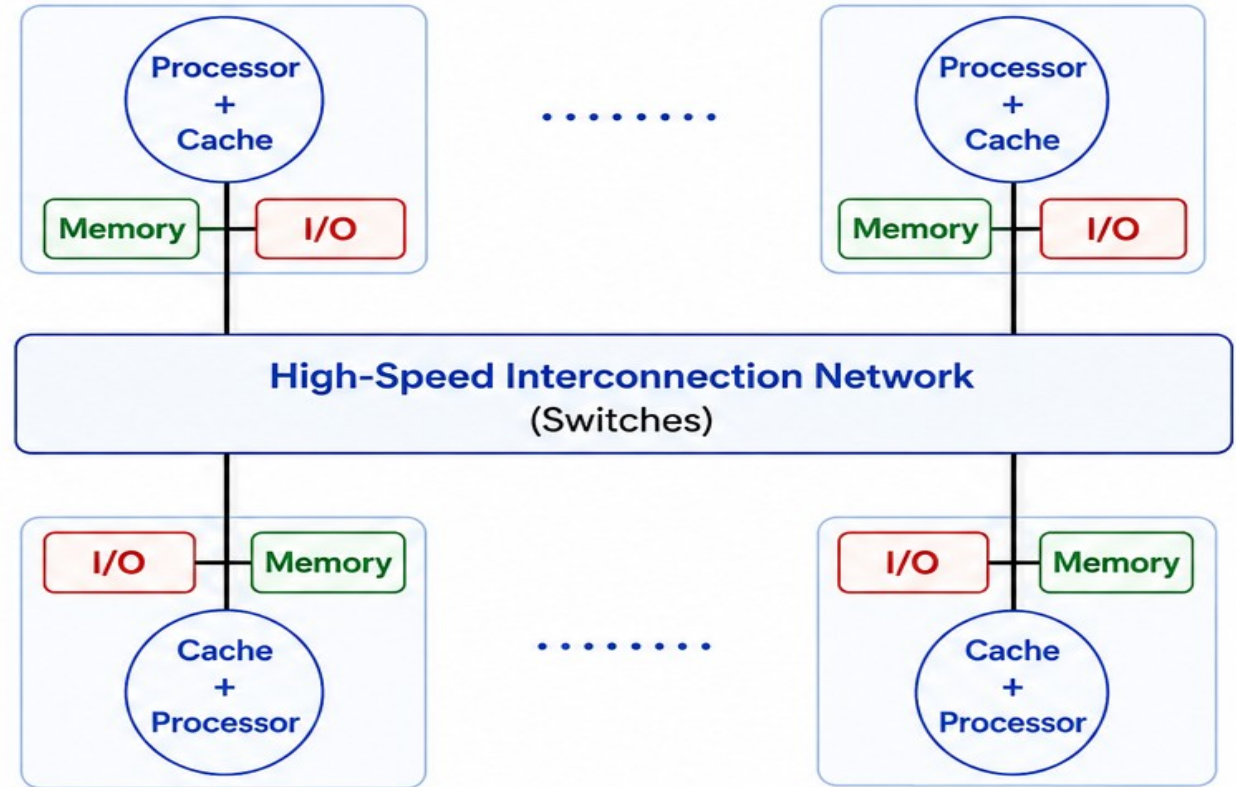


Memory is **physically distributed** among the processors for **better bandwidth**.



Connected through **high-speed interconnection**

e.g., switches.



Performance Characteristics

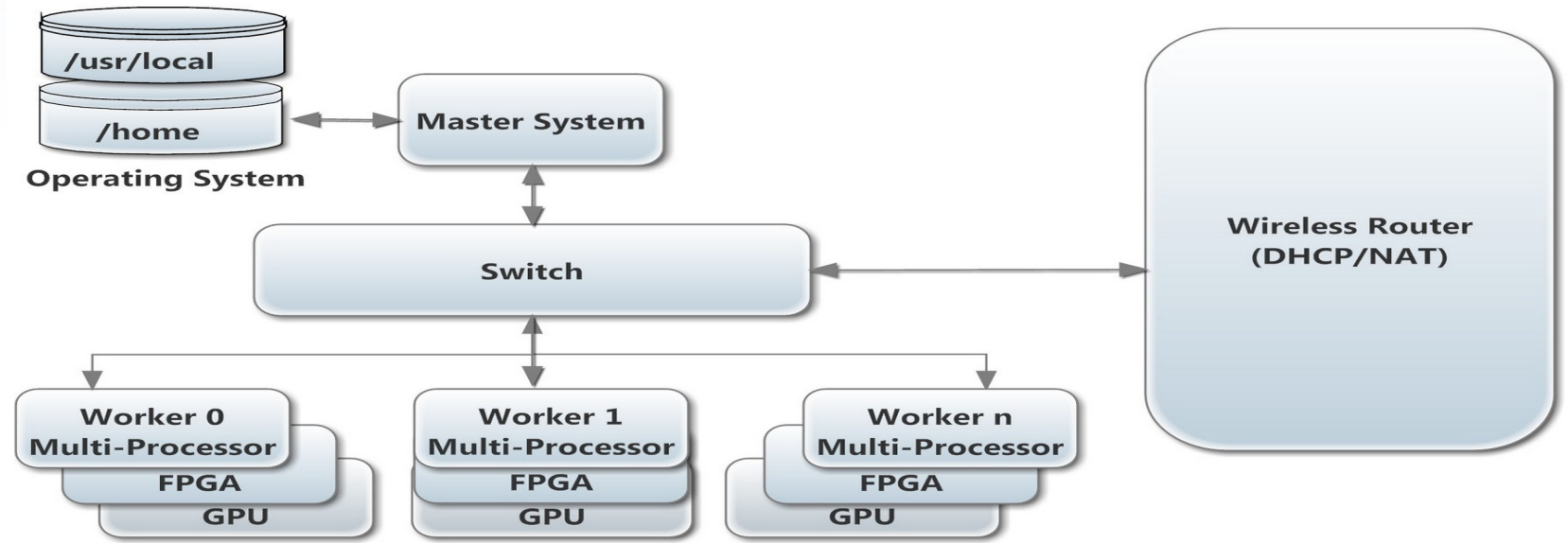
- ✓ High bandwidth for local memory access
- ✓ Low latency for local memory access



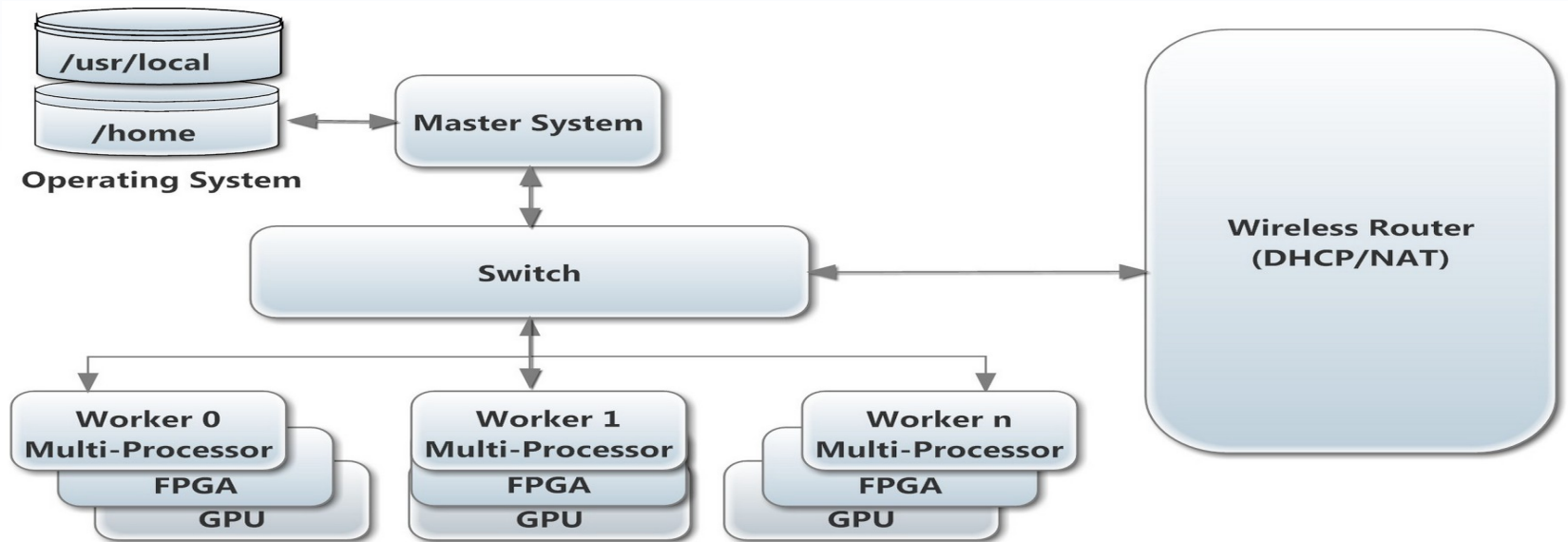
Access to Remote Data

Access to data present in the local memory of some other processor is **complex** and of **high latency**.

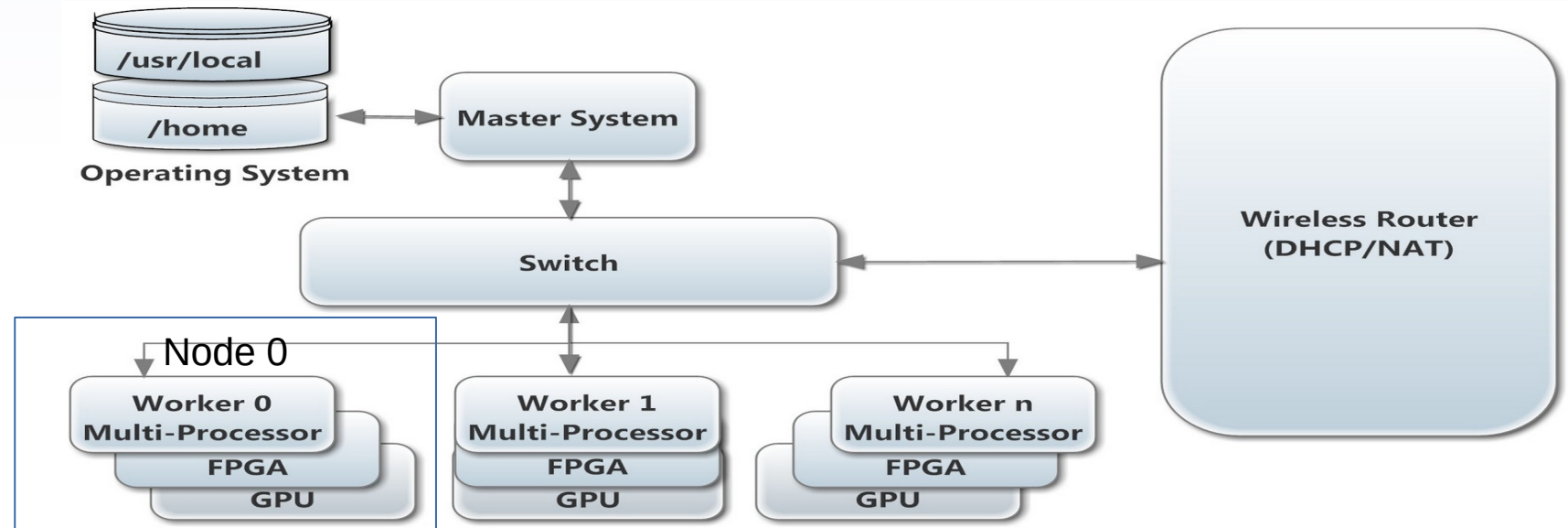
Cluster Computing



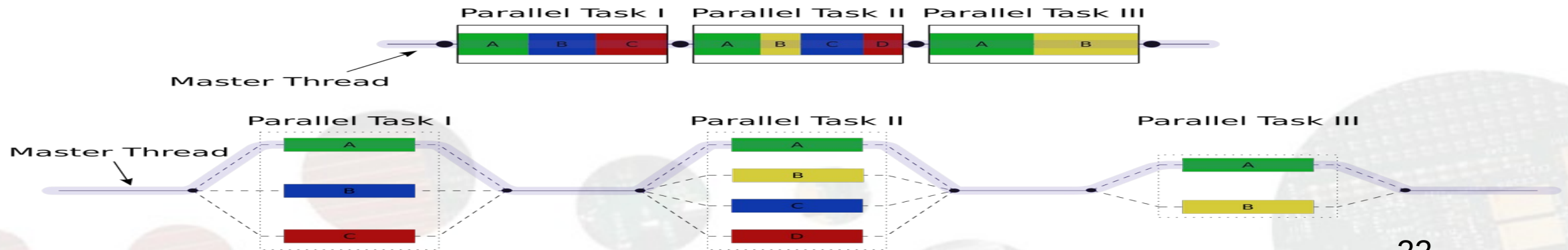
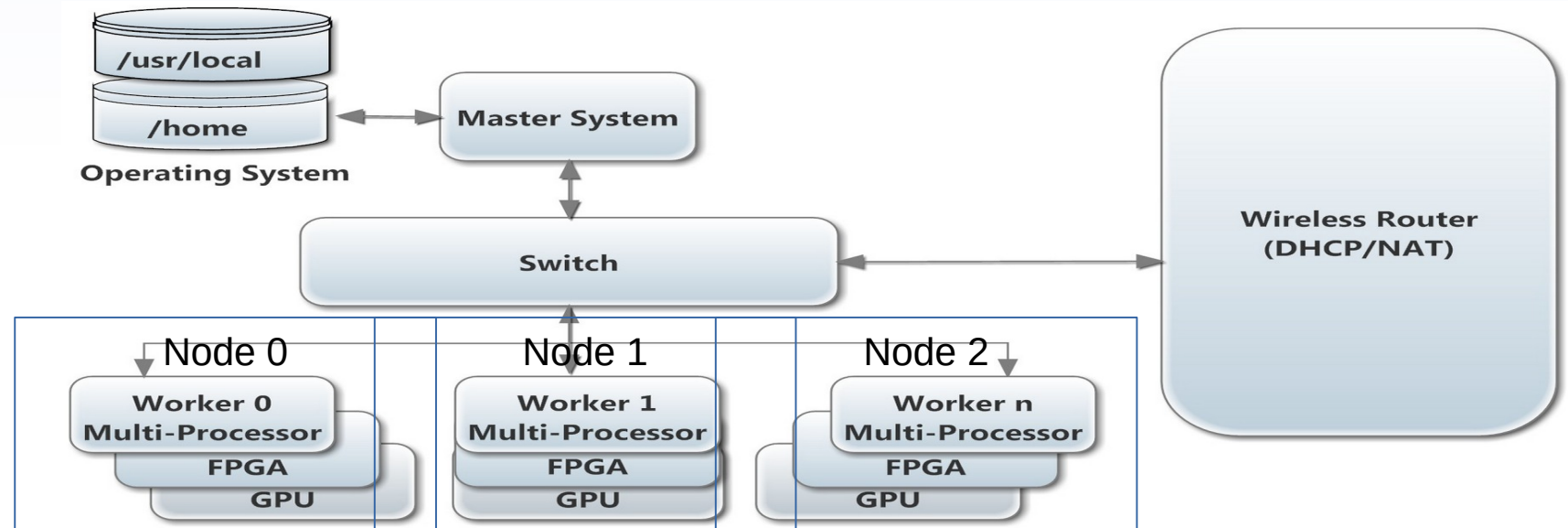
Basic Arch of Supercomputing



Basic Arch of Supercomputing

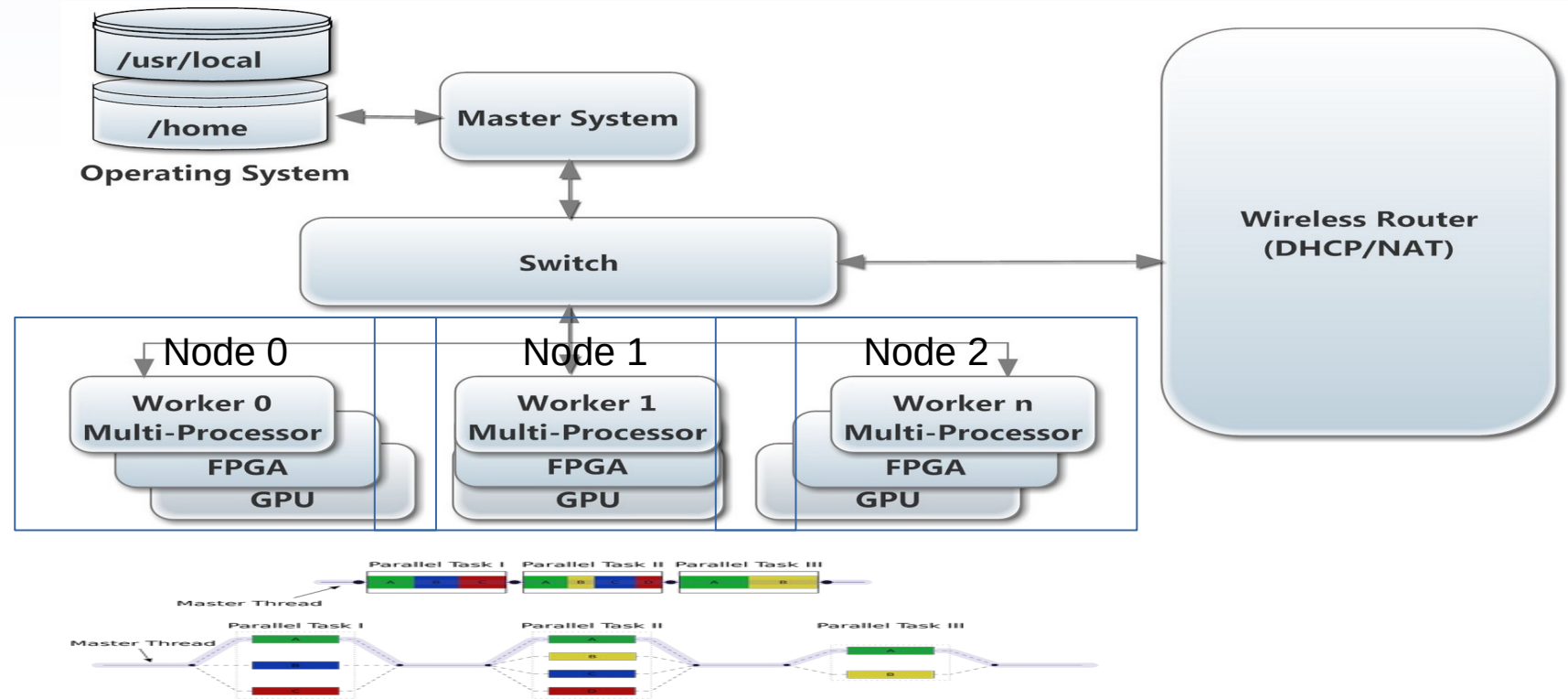


Basic Introduction of Supercomputing



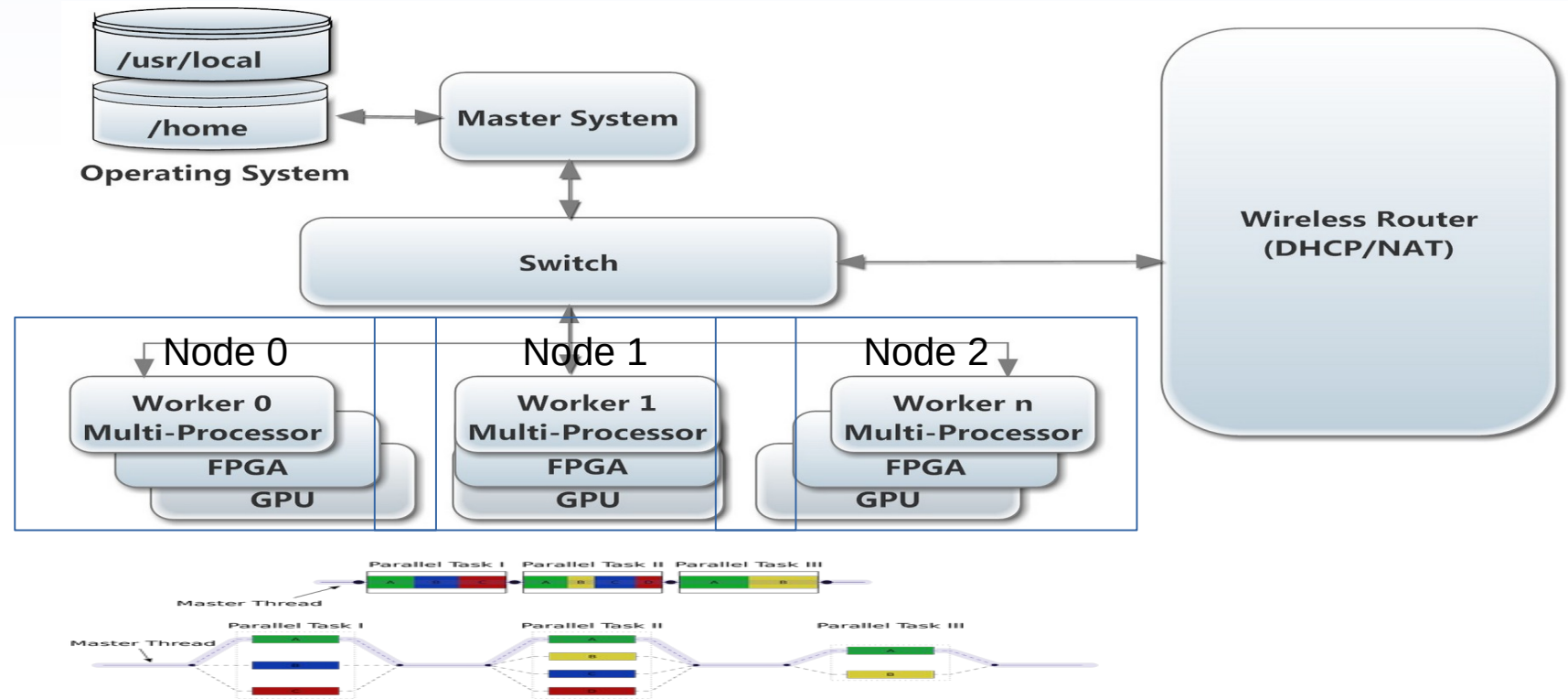
Constituents

- Processing
- Software
- Programming Models
- Storage
- Network
- Accessibility
- Power



Basic Introduction of Supercomputing

- Performance
- Programmability
- Portability
- Scalability
- Accessibility
- Power
- Cost



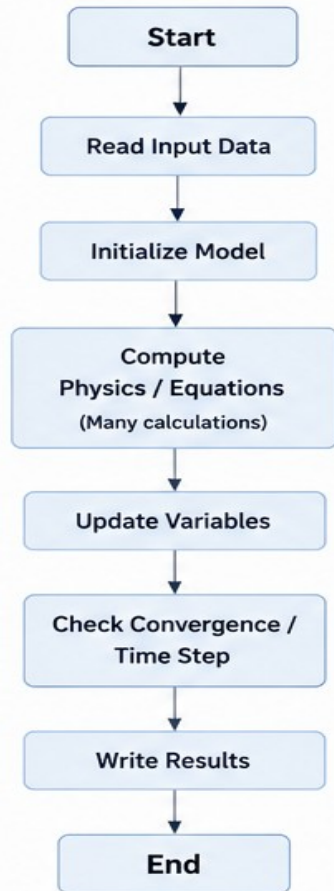
FLOPS = Floating Point Operation Per Second
KFLOPS = 10^3
= One Thousand Computation Per Second
= $12.00 * 1212.222 * 21212 + 232323 \dots$

MFLOPS = 10^6 Million Computation Per Second
GFLOPS = 10^9 Billion
TFLOPS = 10^{12} Trillion
PFLOPS = 10^{15} Quadrillion
EFLOPS = 10^{18} Quintillion
ZFLOPS = 10^{21} Sextillion

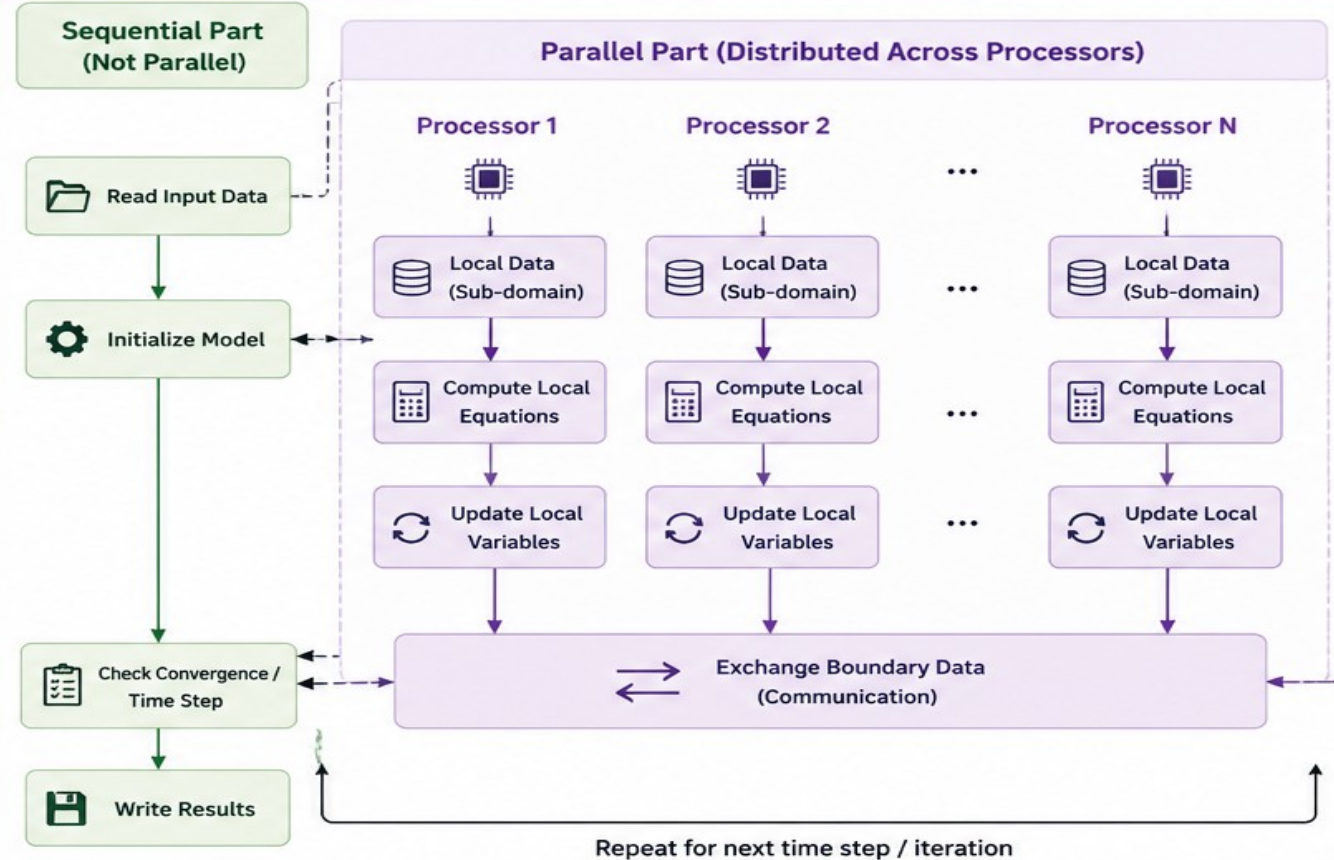
How a Model is Parallelized and Executed

A model is divided into independent parts that can run at the same time. **Some parts must remain sequential.**

1. Full Model (Sequential View)



2. Parallel Execution (Distributed View)



3. What Runs in Parallel and What Does Not?

Runs in Parallel

- Mesh / domain sub-parts
- Physics calculations
- Element / cell computations
- Local variable updates
- Most of the heavy computation

Remains Sequential

- Read / Write I/O
- Model initialization
- Time step control & convergence check
- Global decisions & logging

Key Takeaway

The compute-intensive part of the model is **distributed and executed in parallel**. The control, I/O, and decision-making parts **remain sequential**.



Processor



Local Data



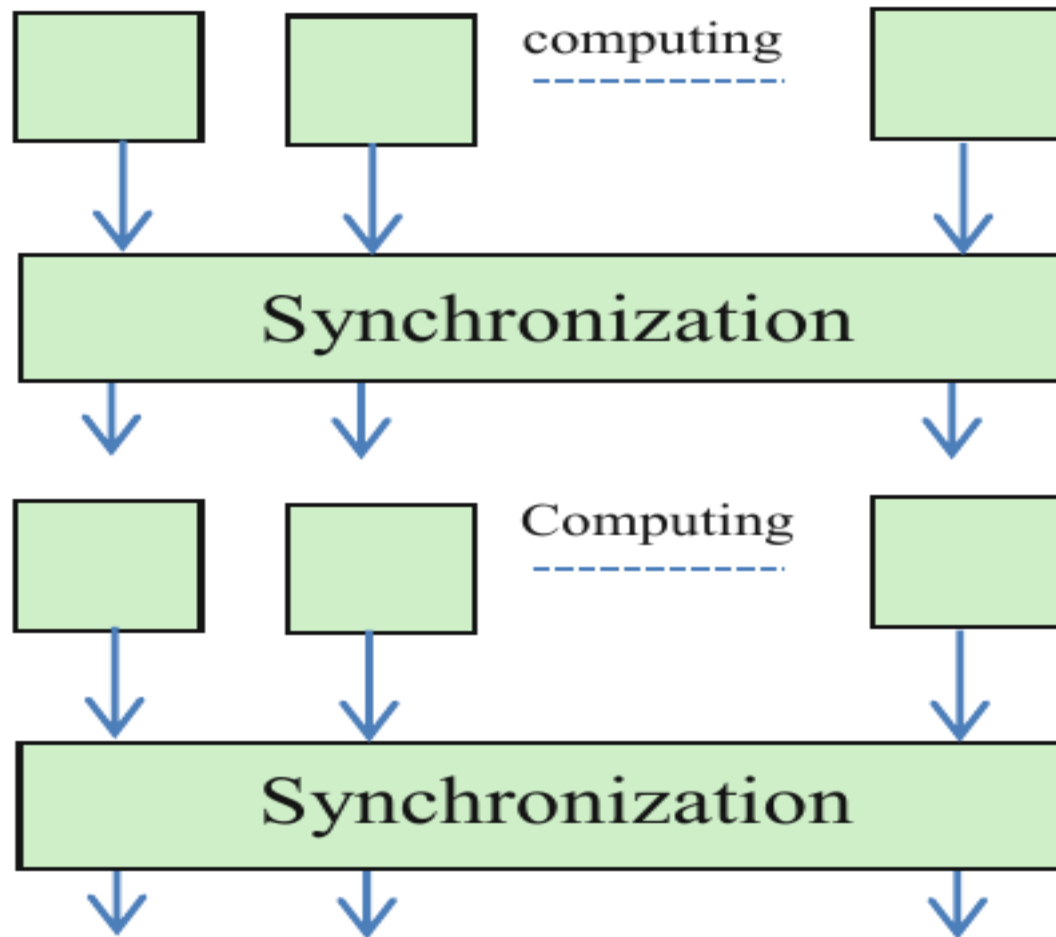
Communication



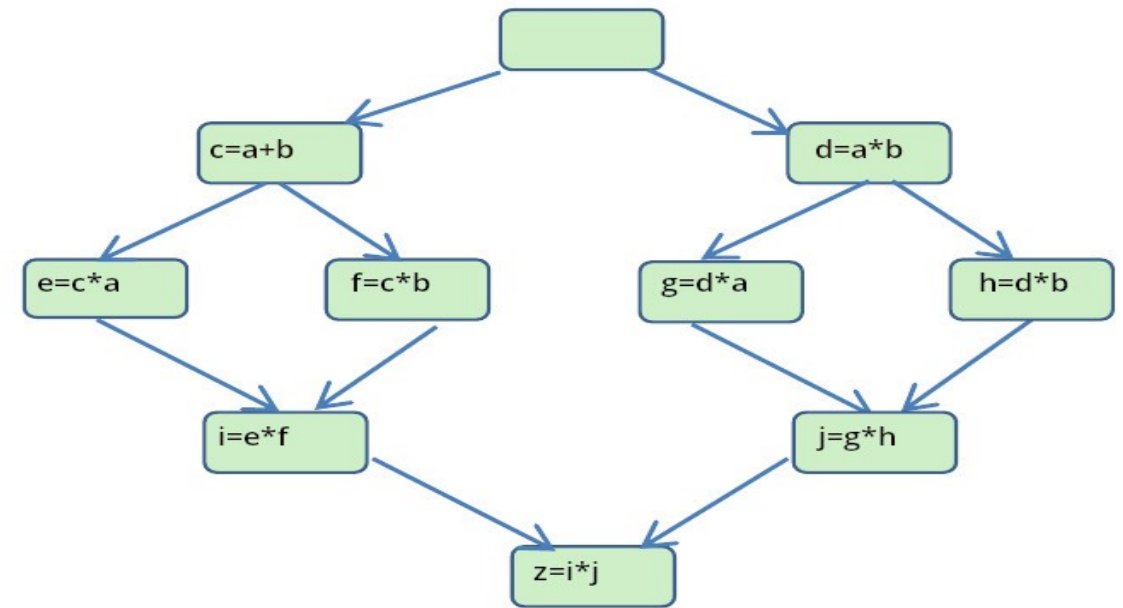
Sequential (Not Parallel)



Parallel (Distributed)



Development and Application of Supercomputing

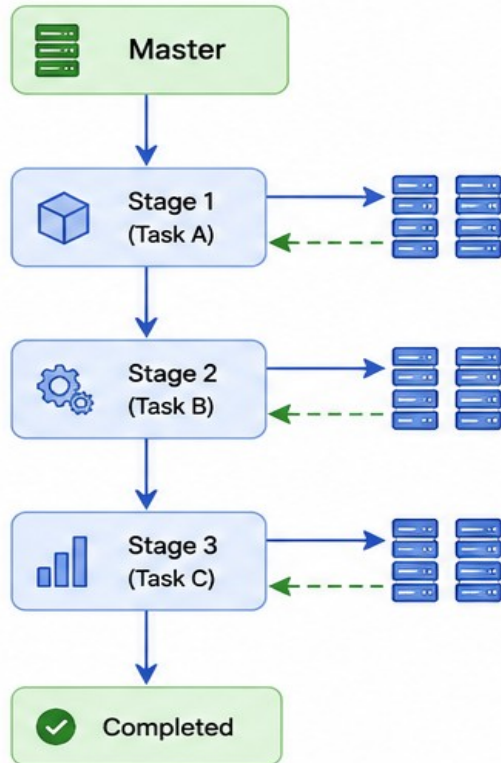


How HPC Cluster Executes Work

Master node distributes tasks to worker nodes and collects results.

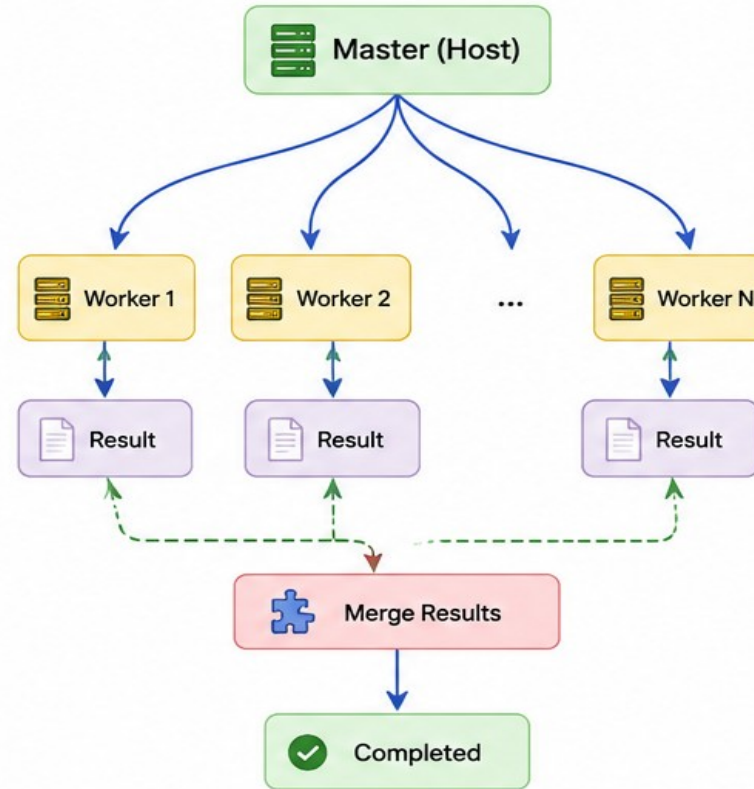
1. Macro-Pipeline

Sequential stages executed in order



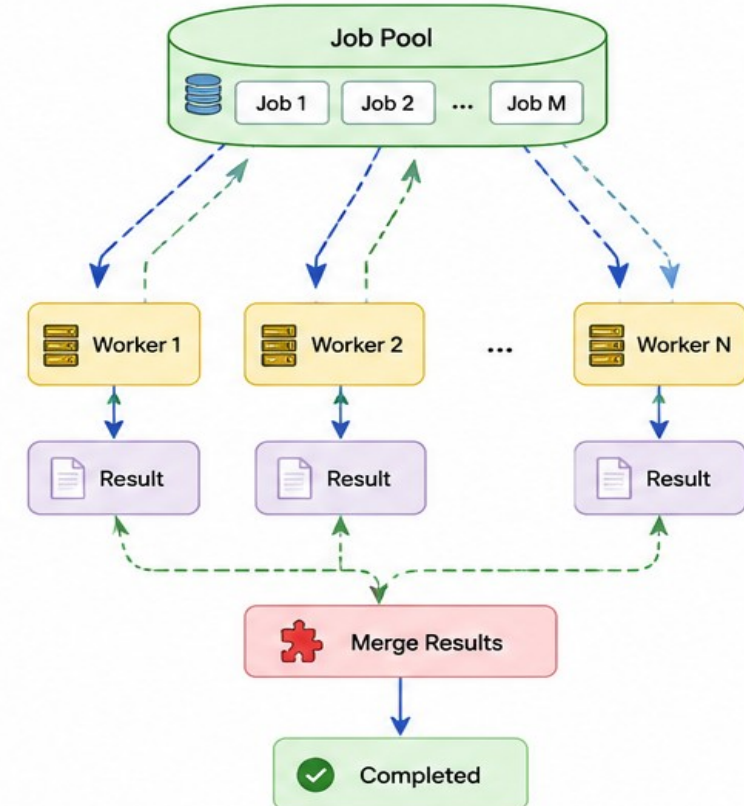
2. Master-Slave

Master assigns tasks; workers process and return results



3. Job-Pool

Workers pull jobs from a shared pool



→ Distribute / Assign Task - - - -> Return Result - - - -> Pull Job (Request)

Master / Host Worker Node Result / Output Merge / Aggregate

What Is High-Performance Computing Cluster?

A coordinated system designed to solve computationally demanding problems efficiently.

HPC

**Compute + Memory +
Network + Storage +
Parallel Software +
Scheduler**

Purpose: reduce time-to-solution or
enable a problem size that cannot fit on
one machine.

Performance

Finish a fixed scientific workload faster.

Capacity

Run a larger problem that needs more memory or processors.

Throughput

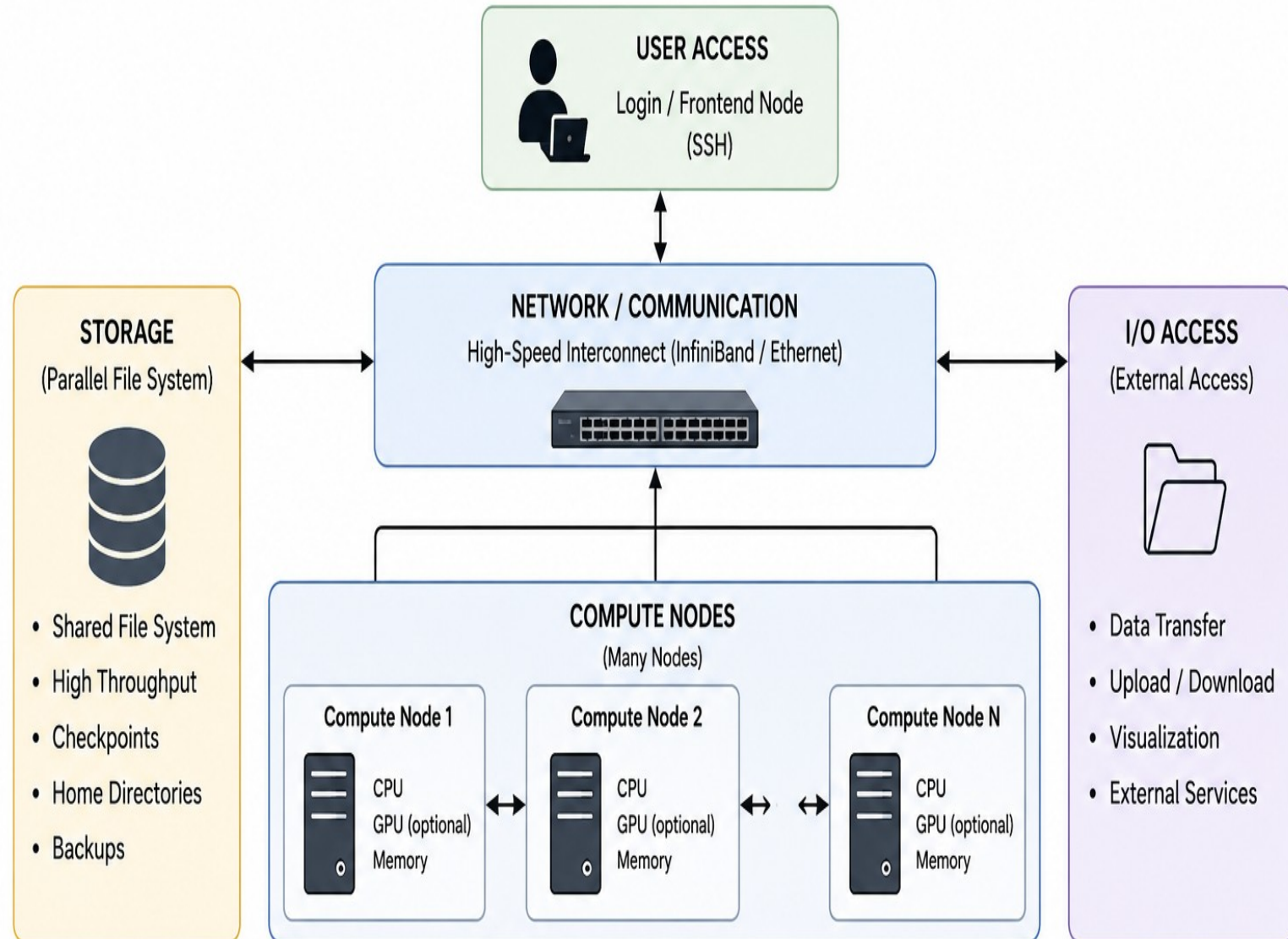
Run many independent simulations or datasets.

Productivity

Shared environment, queues, software and reproducibility.

Components of a Supercomputing System

- Compute Nodes
- Login/Access Nodes
- I/O Nodes
- Network Nodes (Switches/Routers)
- Management Nodes
- Storage Nodes
- Service Nodes



Compute Nodes:

These nodes perform the actual computational tasks and simulations.

Equipped with powerful CPUs and GPUs for parallel processing.

The number of compute nodes can vary based on the expected workload and desired level of parallelism.

Login/Access Nodes:

Provide remote access for users to connect to the cluster.

Users log in to these nodes to submit jobs, manage files, and interact with the cluster.

Not meant for computation but facilitate user interaction.

I/O Nodes:

Handle data input/output operations efficiently.

Optimize data access and transfer between compute nodes and storage systems.

Reduce I/O bottlenecks and latency for data-intensive applications.

Network Nodes (Switches/Routers):

High-speed interconnects (e.g., InfiniBand, Ethernet with RDMA) enable fast communication between compute nodes.

Low-latency, high-bandwidth communication is crucial for parallel processing and data sharing.

Management Nodes:

Host cluster management software, schedulers, and monitoring tools.

Manage resource allocation, job scheduling, system health monitoring, and overall cluster administration.

Storage Nodes:

Manage the storage infrastructure of the cluster.

Provide access to high-performance storage systems, such as parallel file systems or distributed storage solutions.

Store large datasets and simulation results.

Service Nodes:

Host essential services for cluster operation and user interaction.

Services may include LDAP authentication, DNS, NTP (Network Time Protocol), and other services needed for system stability.

Compute Node Interconnect:

High-performance network interconnects between compute nodes facilitate communication for parallel processing and distributed computing.

HPC Classes

General Purpose

vector processing machines - the same operation carried out on a large amount of data simultaneously

Tightly connected cluster computers (NUMA) - communication oriented architectures engineered from ground up, based on high speed interconnects and large number of processors

Commodity clusters - collection of large number of commodity PCs (COTS) interconnected by high-bandwidth low-latency network

Special Purpose

high performance computing devices with a hardware architecture dedicated to solve a single problem (equipped with custom ASICs or FPGA chips)

PROFESSIONAL HPC FOR MODELING & SIMULATION

Accurate Results • Reliable Computation • High Performance at Scale

1. HARDWARE REQUIREMENTS

COMPUTE



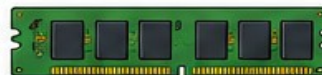
- CPU for control & logic
- GPU for massive parallel computations

NUMERICAL PRECISION

FP64 (64-bit)
High accuracy
For scientific simulations

FP32 (32-bit)
Higher speed
For less sensitive tasks

MEMORY



- Large Capacity
- High Bandwidth
- Low Latency

ECC MEMORY
Detects and corrects
bit errors

INTERCONNECT



- High Bandwidth
- Low Latency
- RDMA Support

STORAGE / I/O



- Fast Parallel File Systems
- High Throughput
- Checkpoints & Data

RELIABILITY & AVAILABILITY



- ECC Memory
- Hardware Monitoring
- Fault Tolerance
- Stable Power & Cooling

2. SOFTWARE SUPPORT

SYSTEM SOFTWARE



- Linux OS
- Compilers
- Libraries
- Drivers

PARALLEL PROGRAMMING

MPI

OpenMP

- MPI for distributed memory (across nodes)
- OpenMP for shared memory (within node)

ACCELERATION & LIBRARIES



OpenACC



- Math Libraries (BLAS, LAPACK)
- FFT, Sparse Solvers, etc.
- GPU Acceleration

APPLICATION SOFTWARE



- CFD, FEA, MD, Quantum
- Climate, Weather, etc.
- In-house Codes

JOB MANAGEMENT



- Job Scheduling
- Queues & Priorities
- Resource Allocation

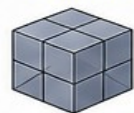
DEVELOPMENT & TOOLS



- Debuggers
- Profilers
- Performance Tools
- Version Control

3. HOW IT WORKS TOGETHER

Problem



Define geometry, physics, boundary conditions

Discretization



Mesh / particles / equations

Parallel Execution



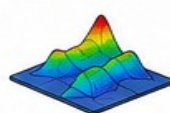
Run on many CPU cores / GPUs across nodes

Data & I/O



Read / Write checkpoints and results

Post Processing



Analyze, visualize, validate results

Results



Accurate, reliable, reproducible

4. WHAT MATTERS MOST



ACCURACY

Use proper precision (FP64 when needed) and stable algorithms.



RELIABILITY

ECC memory, fault tolerance and monitoring ensure correct long runs.



PERFORMANCE

Balance compute, memory, storage and network to remove bottlenecks.



SOFTWARE ECOSYSTEM

Compilers, libraries, MPI/OpenMP, tools and applications are essential.



SCALABILITY

Good performance as the problem and hardware grow.

5. HPC CLUSTER ARCHITECTURE (EXAMPLE)

Login Nodes



Access & submit jobs

Compute Nodes



CPU + GPU
High memory, ECC

High-Speed Network



InfiniBand / HDR
RDMA

Parallel Storage



Parallel File System
High throughput

Backup



Backups & Archive



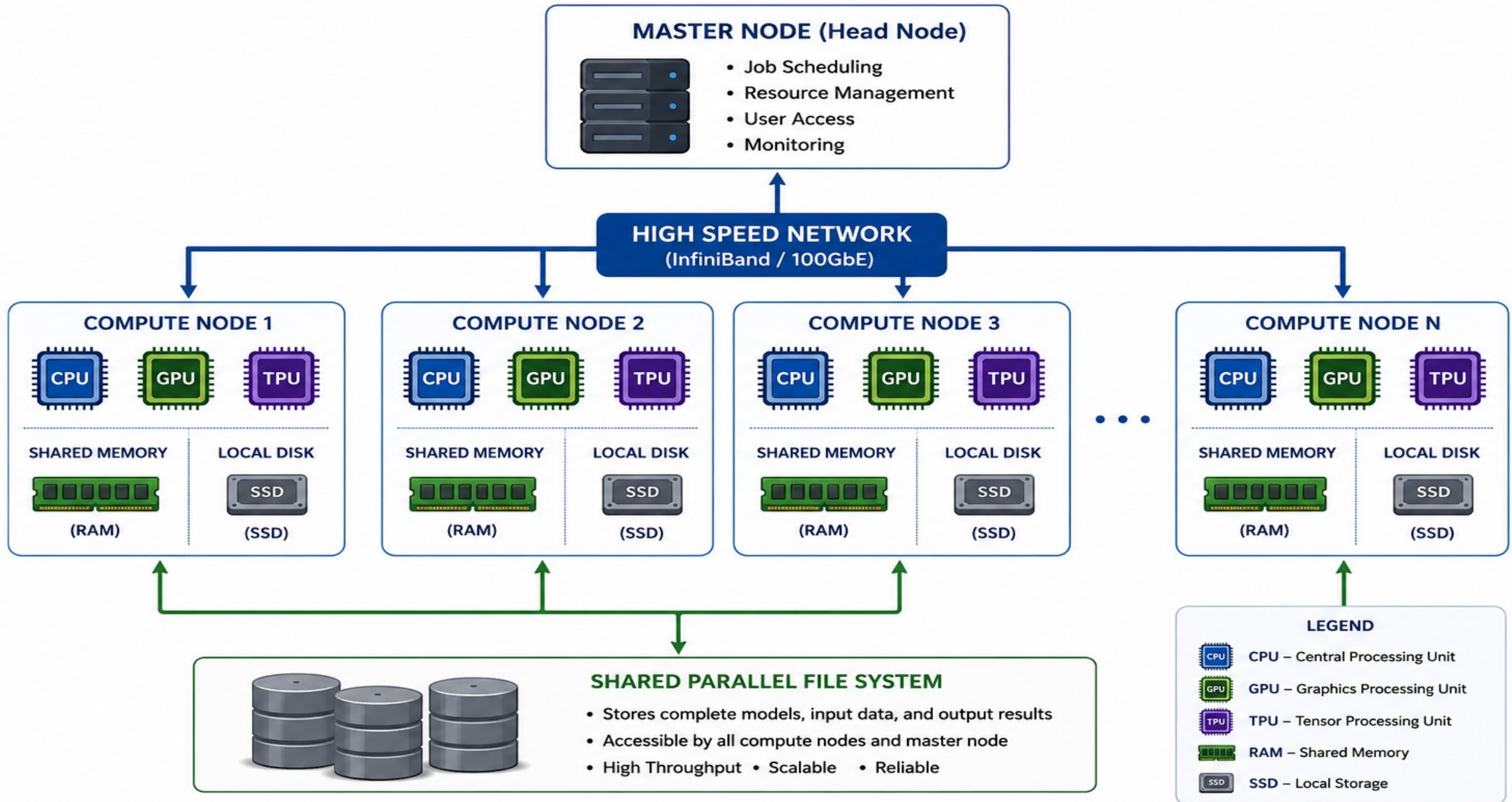
BOTTOM LINE

Professional HPC combines the right hardware, reliable architecture and strong software so scientists can run large simulations and obtain accurate, trustworthy results.

Commodity HPC Cluster

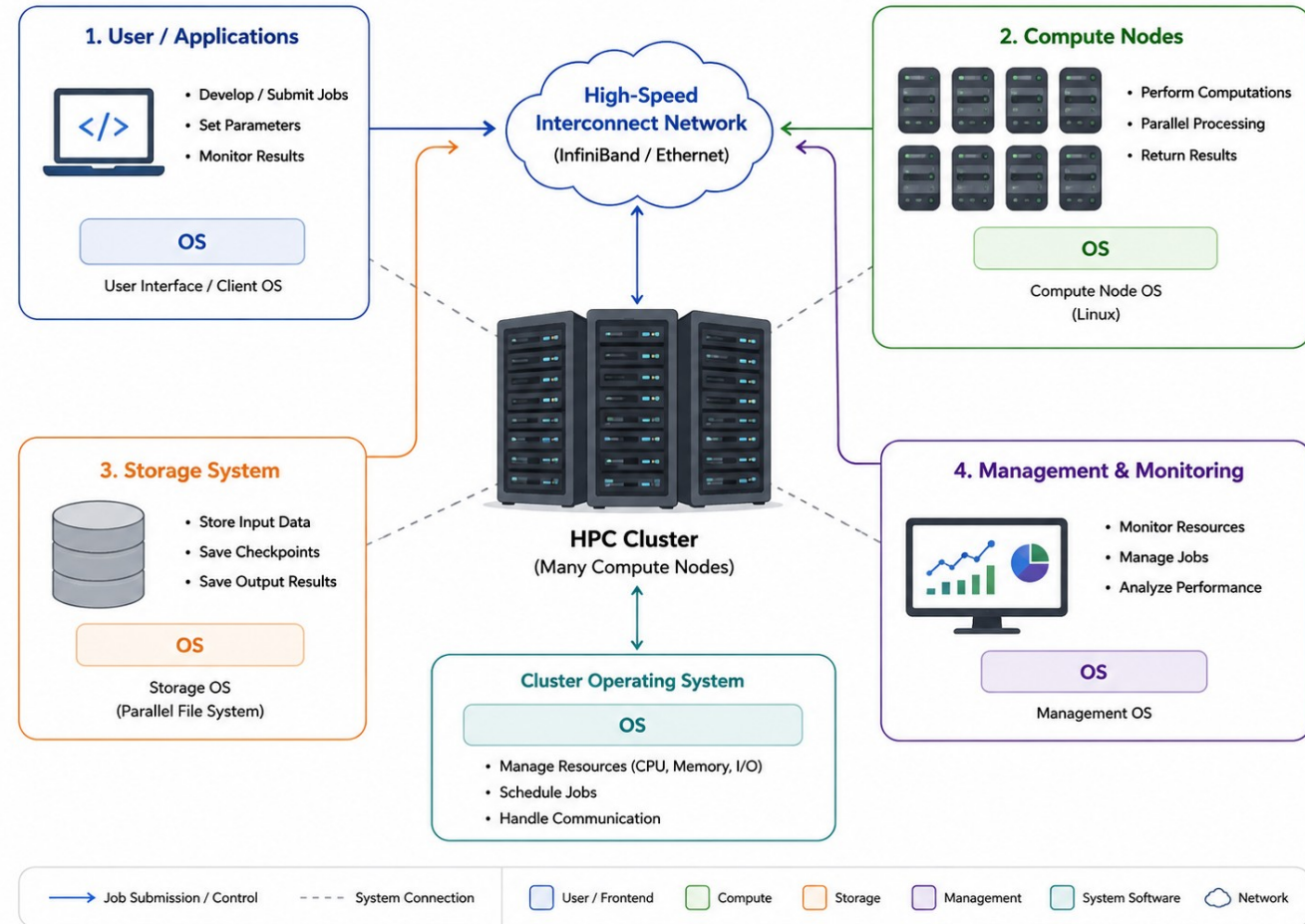
Cluster is a collection of connected, independent computers working in unison to solve a problem COTS technology nodes are interconnected by Ethernet LAN, Myrinet, QsNet ELAN etc. computation can be performed by using popular programming toolkits and frameworks: OpenMP, MPI clusters require dedicated management software.

HPC CLUSTER ARCHITECTURE



Basics of Supercomputing System

- Operating System
- Network
- Storage Management
- Software Stack Echo-System



Operating System in an HPC Cluster

The Operating System (OS) is the core software layer that manages and coordinates all resources in an HPC cluster so scientific applications can run efficiently and reliably.

What the OS Does



Resource Management

Manages compute nodes, CPUs, memory, GPUs, storage and network resources.



Scheduling

Decides which job runs where and when to maximize utilization and fairness.



Virtualization

Provides isolation and abstraction of resources (containers / VMs) to run multiple workloads.



Orchestration

Coordinates nodes, services and workflows across the cluster.



Security & Access

Manages users, groups, permissions and secures data and resources.



Monitoring & Control

Monitors system health, performance and handles alerts and failures.

Operating System



- Kernel & system services
- Process & resource management
- Drivers (CPU, GPU, NIC, Storage)
- File systems & memory management
- Job management & scheduling interface

Network



- High-speed interconnects
- Network stack & protocols
- Data transfer & communication
- RDMA / MPI transport support

Storage Management



- File systems (Lustre, GPFS, NFS)
- Storage allocation & quotas
- Data caching & buffering
- I/O scheduling & optimization
- Data protection & backups

Parallel Programming Models



- MPI (Message Passing Interface)
- OpenMP (Shared Memory)
- CUDA / HIP (GPU Programming)
- Hybrid models (MPI + OpenMP)
- Libraries & runtime support

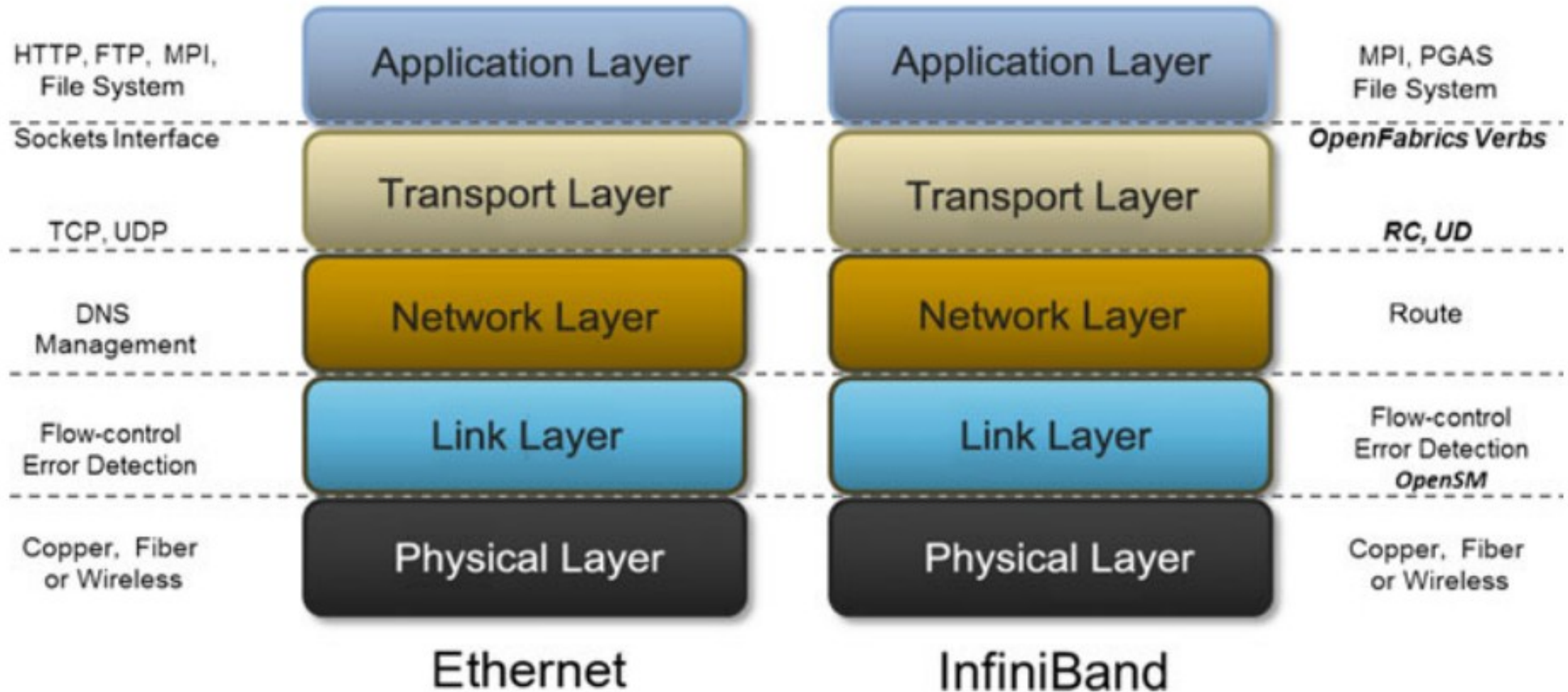


Overall Purpose

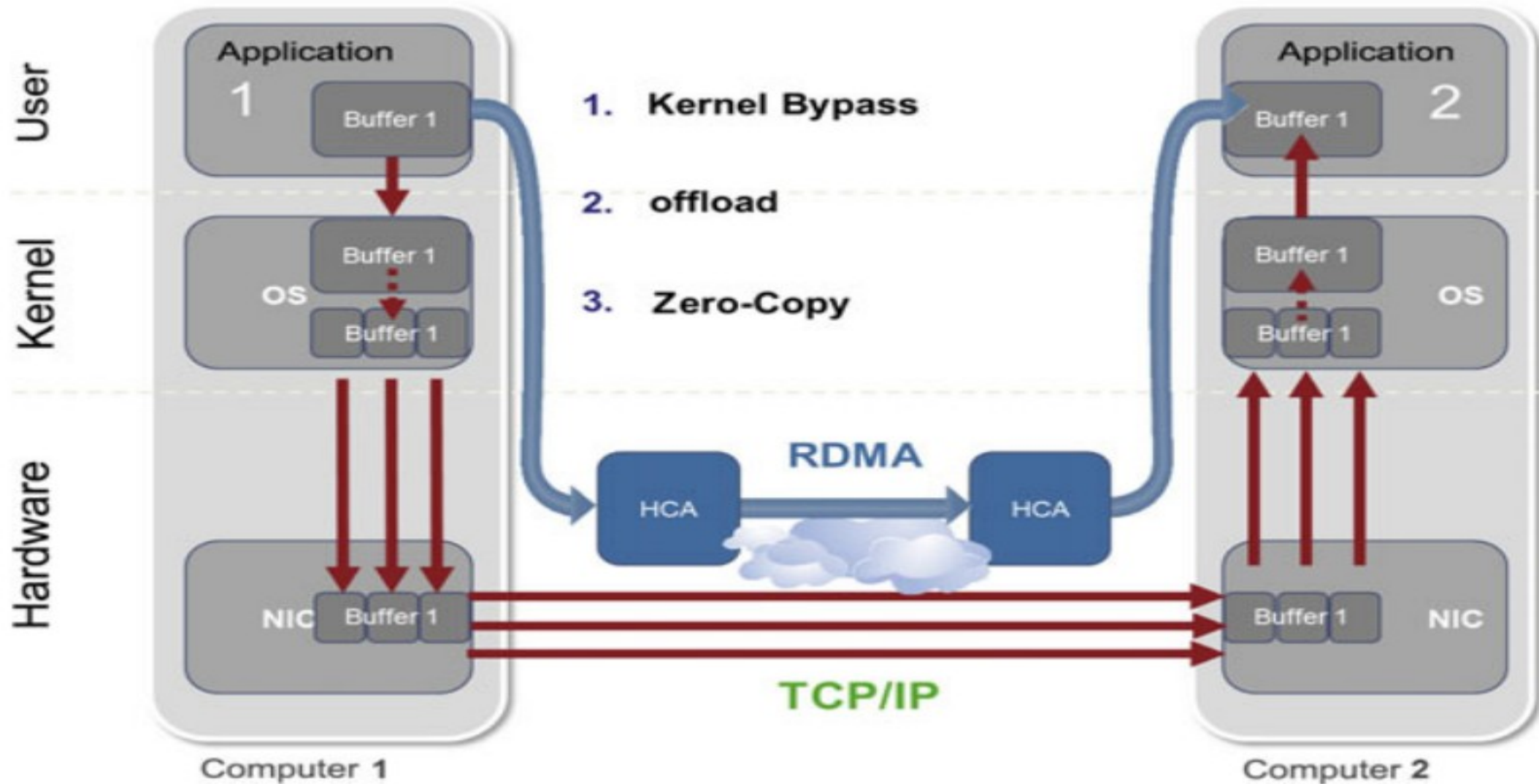
The OS in an HPC cluster ensures efficient resource utilization, high performance, scalability, reliability and ease of use for scientific and engineering workloads.



Network Protocol



TCP/IP and Remote Direct Memory Access



Why the Interconnect Matters

Distributed simulations must exchange data between nodes.

LATENCY

How long does a small message take to start moving?

Important for frequent synchronization and many small messages.

BANDWIDTH

How much data can be moved per second?

Important for large messages, halo exchange, checkpointing and distributed arrays.

TOPOLOGY

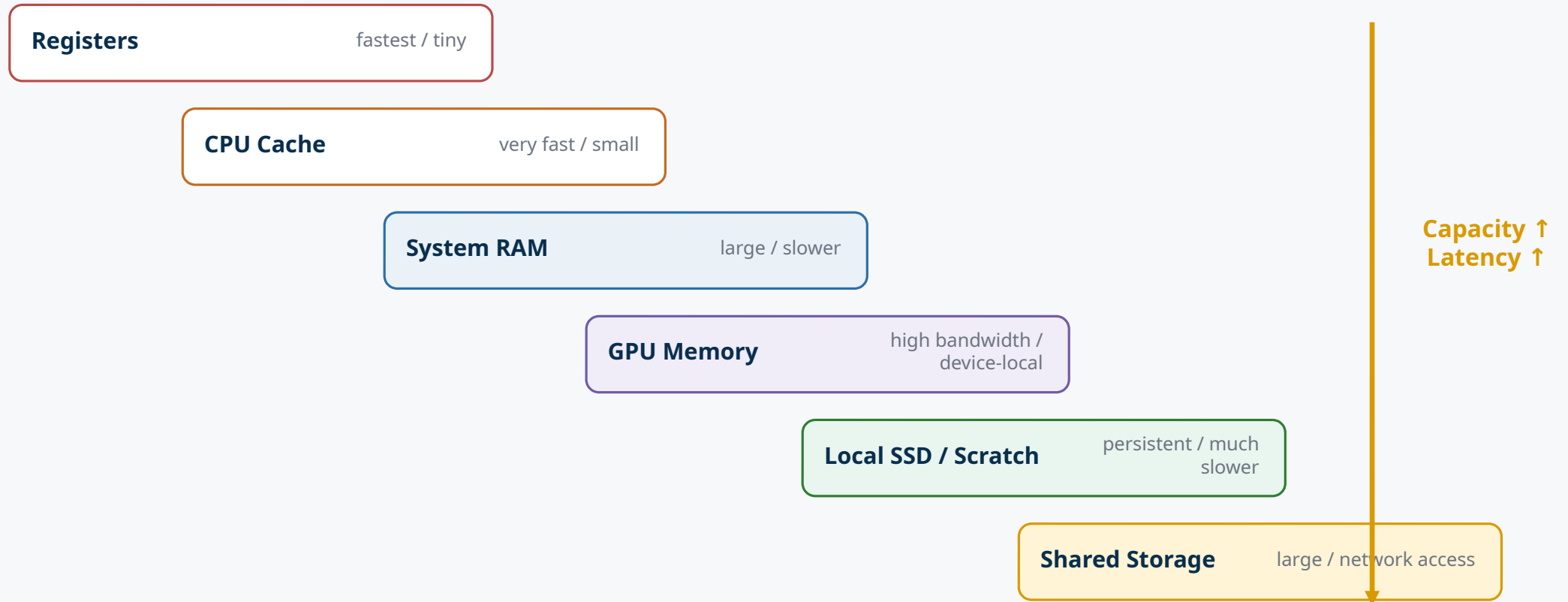
How are nodes physically connected?

Determines contention and communication paths as the cluster scales.

A fast CPU cannot compensate for a slow network when the algorithm spends much of its time communicating.

Memory Hierarchy: Fast Is Small, Large Is Farther Away

A simulation can be compute-bound or memory/data-movement-bound.



Performance rule: moving data is often more expensive than performing arithmetic on data that is already nearby.

Storage System Classes – At a Glance

DAS

(Direct Attached Storage)

Storage is directly connected to a single server.

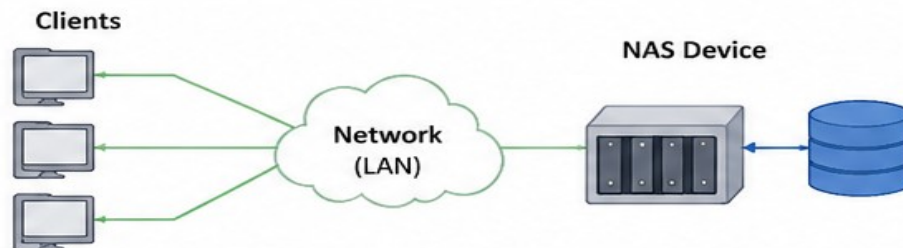


- ✓ Simple
- ✗ Not shared

NAS

(Network Attached Storage)

Storage is on a network device and accessed over the network using file protocols.



- ✓ Easy sharing
- ✓ File-level access
- ✗ Lower performance than SAN

SAN

(Storage Area Network)

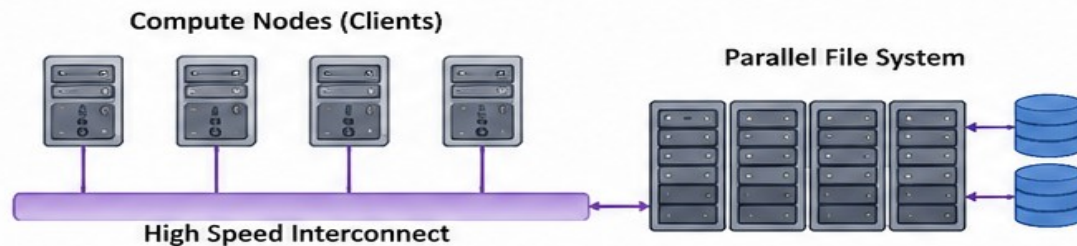
Storage is on a dedicated network and presented as block-level storage to servers.



- ✓ High performance
- ✓ Block-level access
- ✗ More complex & expensive

Parallel File System Storage (Cluster / High Performance Storage)

Distributed storage across many servers/nodes for parallel access and very high performance.



- ✓ Massive performance
- ✓ Scalable
- ✓ Built for HPC / AI
- ✗ Complex & costly

Storage and Data Movement

Scientific computing can generate **as much data as it computes**.



GOOD PRACTICES



Right data, right place

Keep active data in appropriate (fast) storage tier.



Checkpoint smartly

Checkpoint long jobs so they can restart from last good state.



Avoid I/O bottlenecks

Avoid writing every possible variable at every time step.



Store what matters

Store only the data needed for analysis and reproducibility.



Key Takeaway

Efficient data movement and storage strategy reduces I/O time, saves storage costs, and accelerates scientific discovery.

Commodity Cluster Software

Operating system - Linux or SSI Linux (Single System Image) Platform for specialized applications for science, engineering and business (simulation, modeling, data mining)

Distributed computation environments are used for software development (OpenMP, MPI)

Common supercomputer applications require porting to clusters

Software Stack Echo-System

- (1) Multi-threading programming languages for the shared memory model, such as POSIX Threads, Java Threads, OpenMP.
- (2) Message passing programming languages for the distributed memory model, such as MPI, PVM.
- (3) Parallel programming languages for data-parallel designs, such as HPF (High Performance Fortran, Hadoop/Spark).
- (4) Parallel programming languages for heterogeneous platforms, such as CUDA developed by NVIDIA.

Namal Supercomputing Facility — ~1.6 PetaFLOPS

20 HPC compute nodes.

Each node composed on average of:

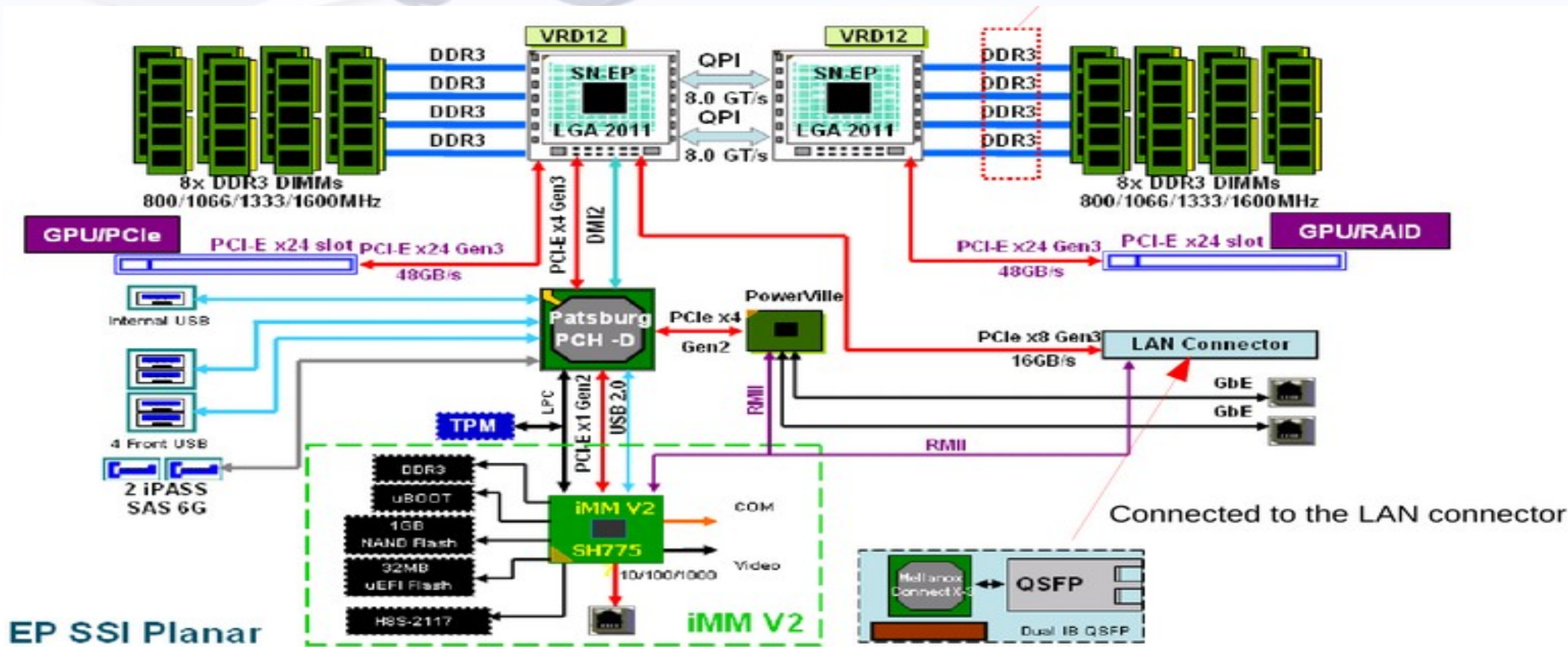
- 80 CPU cores
- 2 × NVIDIA RTX 4070 Ti GPUs
- 256 GB RAM
- 2 TB SSD storage
- 10 GbE cluster interconnect

Total: 1,600 CPU cores • 40 GPUs • 5 TB RAM • 40 TB SSD

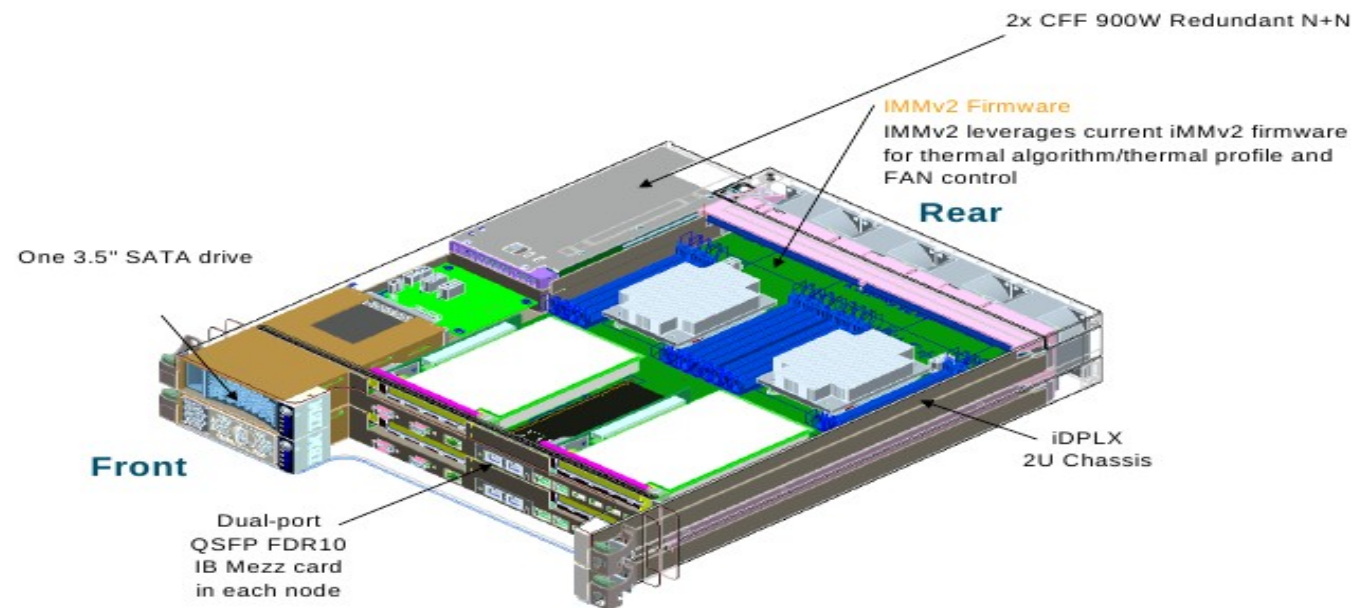
Software: Linux • Slurm • MPI • OpenMP • CUDA • Containers • GROMACS • Quantum ESPRESSO • AI/ML frameworks.



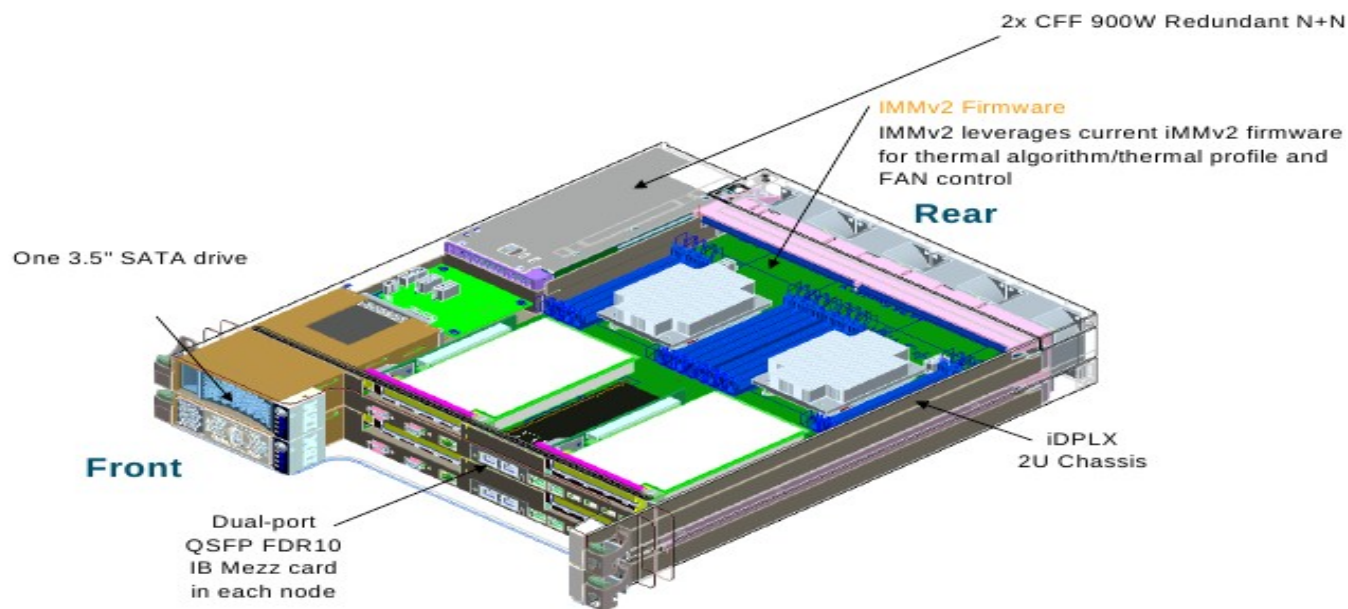
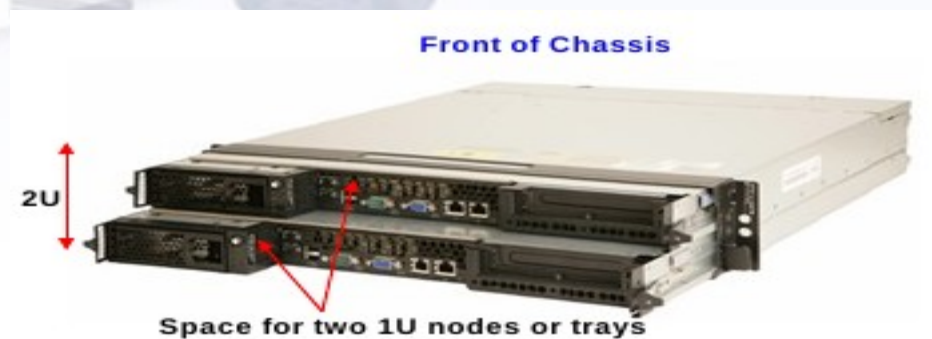
Bottom-up Design Approach



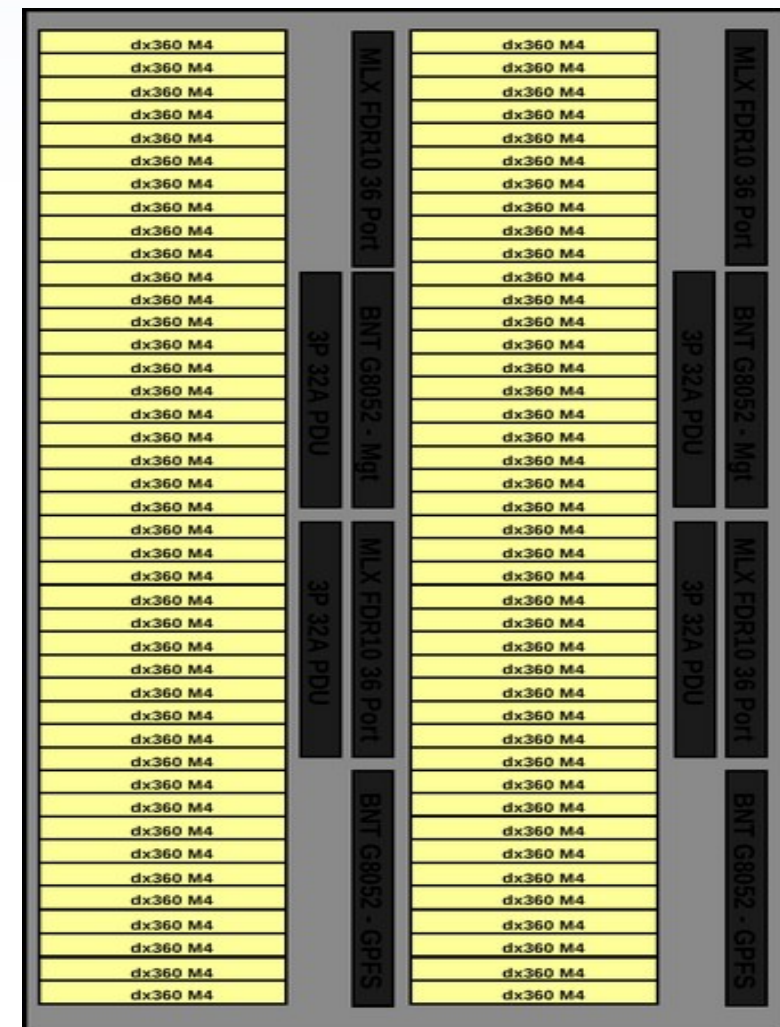
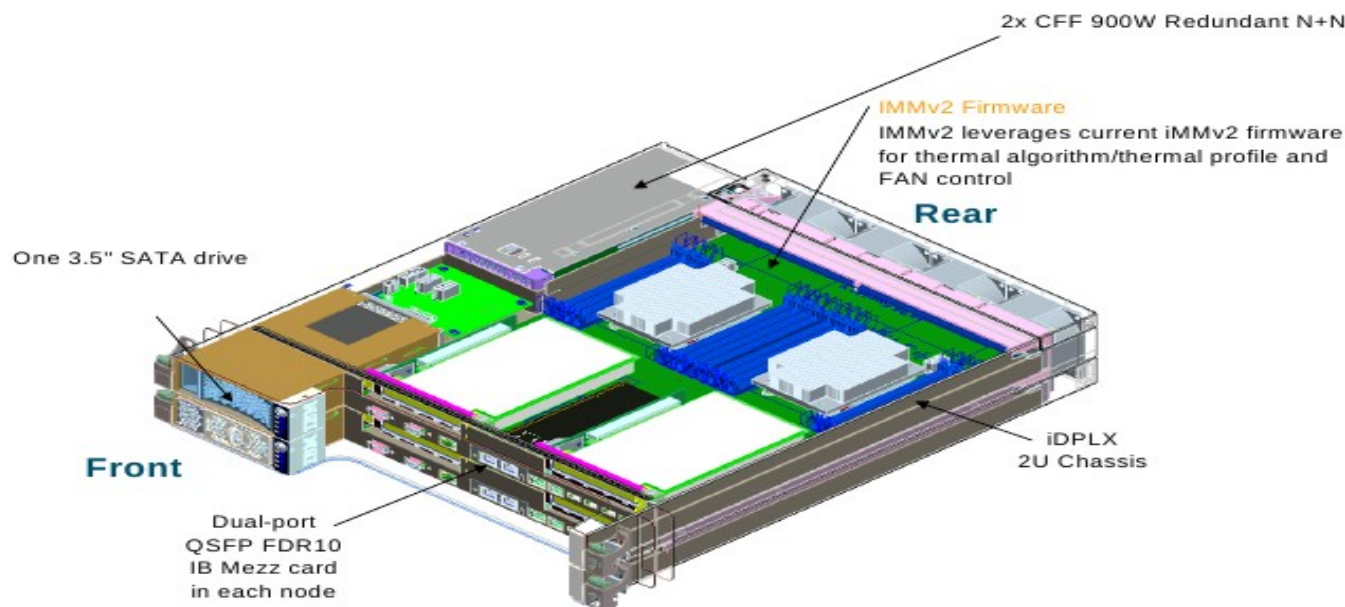
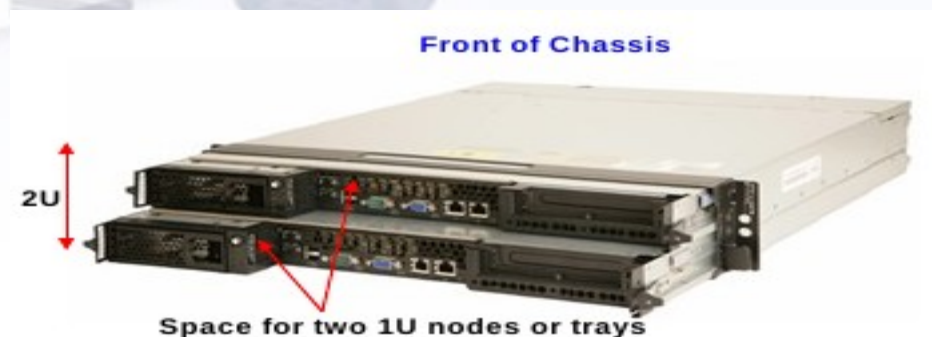
Bottom-up Design Approach



Bottom-up Design Approach

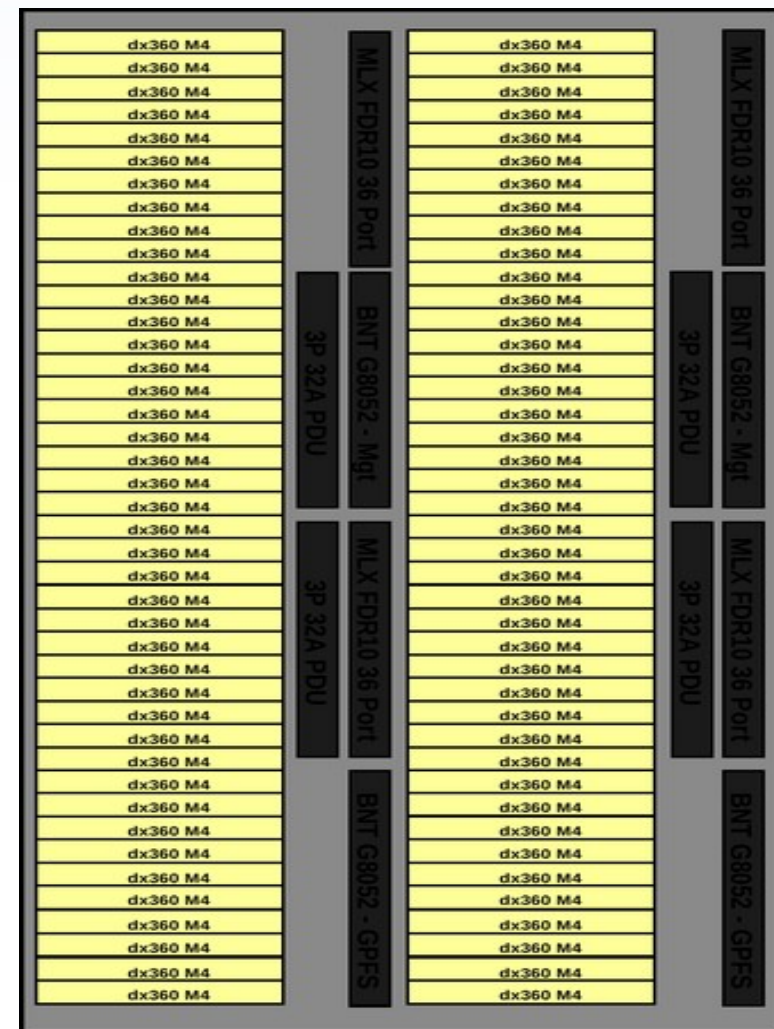
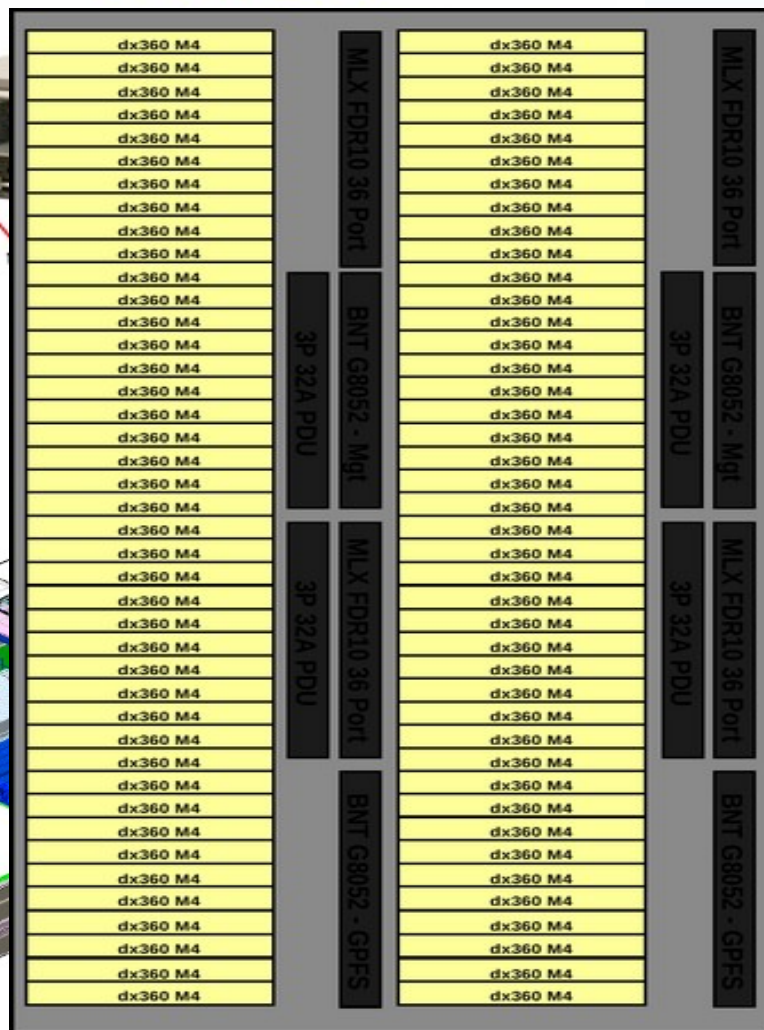
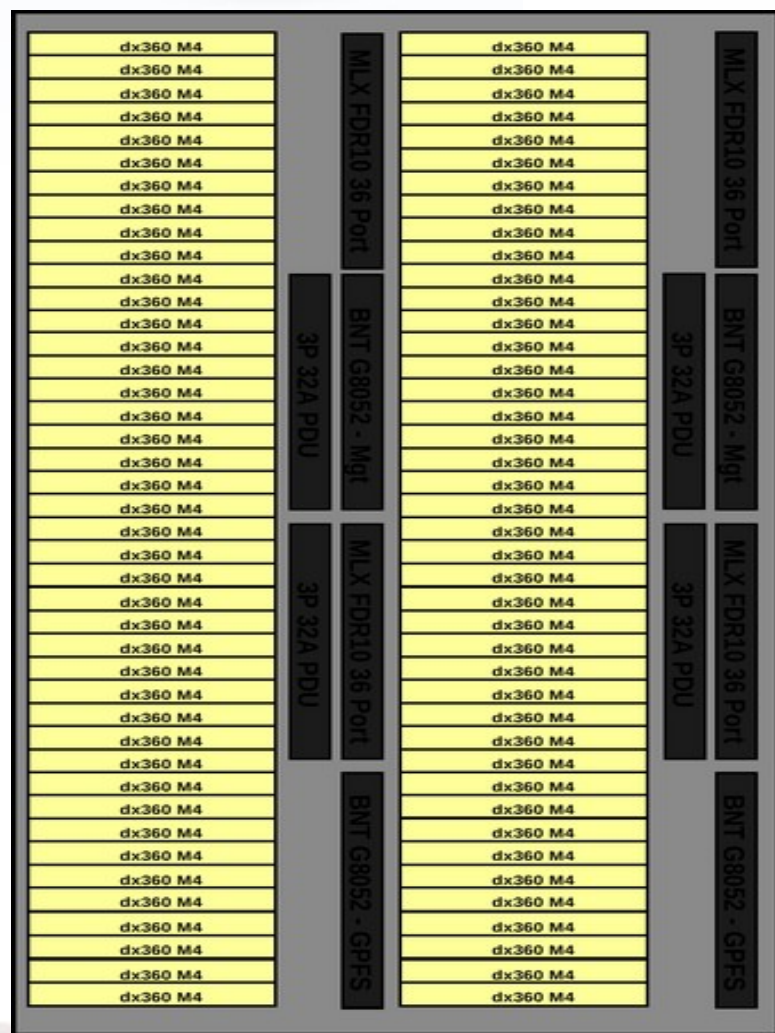


Bottom-up Design Approach



Bottom-up Design Approach

Front of Chassis



What Resources Does a Scientific Job Request?

A good job description matches the application to the hardware.

NODES

Use more nodes only if the code scales across nodes.

CPU CORES

Match ranks/threads to the code configuration.

GPUs

Request GPUs only for GPU-enabled runs.

MEMORY

Estimate from model size and previous runs.

TIME

Set a realistic wall time with some margin.

PARTITION

CPU, GPU, debug, high-memory, etc.

Correct resources = shorter queue + better performance + fairer use of the cluster.

How Do We Measure HPC Performance?

Measure the outcome that matters to the scientific workflow.

TIME-TO-SOLUTION

How long does one simulation take?
Lower is better

THROUGHPUT

How many simulations finish per hour/day?
Higher is better

SPEEDUP

T_1 / T_p
How much faster with p processors?

EFFICIENCY

Speedup / p
How well are processors used?

UTILIZATION

Are CPUs/GPUs busy?
Low utilization signals a bottleneck

COST / ENERGY

Resources per scientific result
Important for sustained operation

Peak hardware FLOPS are useful—but application performance is what determines scientific productivity.

Common Beginner Mistakes on HPC

Avoid these before they cost queue time—or scientific confidence.

Running heavy jobs on the login node

Use the scheduler.

Requesting all GPUs “just in case”

Benchmark the actual need.

Using many MPI ranks without testing

Communication can make it slower.

Ignoring memory limits

Out-of-memory jobs fail abruptly.

Writing huge outputs too frequently

I/O can dominate runtime.

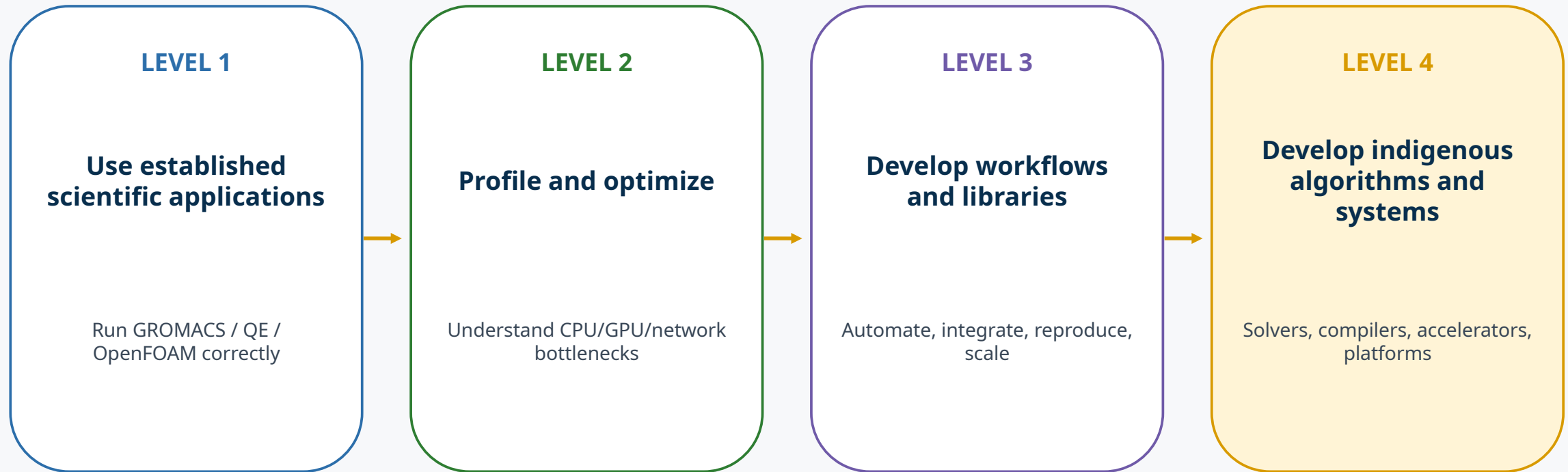
Scaling before validating the model

Fast wrong science is still wrong.

Scientific correctness first → performance second → scale third.

From HPC Users to HPC Developers

National capability grows when we move beyond “running software.”



The long-term objective is not only access to compute—it is the ability to design, optimize and operate the complete computational stack.

Key Takeaways

What to remember before the hands-on HPC session.

- 1 **A node is one server; a cluster coordinates many nodes.**
- 2 **CPU, GPU, memory, network and storage must be balanced.**
- 3 **OpenMP shares memory; MPI distributes work across processes/nodes.**
- 4 **Slurm allocates resources and manages queued jobs.**
- 5 **More processors help only while the application continues to scale.**
- 6 **Profile first; optimize second; scale third.**

NEXT: Hands-on HPC computing with Slurm on the Namal Supercomputing Cluster